

Ένας ταχύτερος ισχυρά πολυωνυμικός αλγόριθμος για το πρόβλημα ροής ελαχίστου κόστους

Νίκος Χατζευφραιμίδης

Διπλωματική Εργασία

Επιβλέπων: Γεωργιάδης Λουκάς

Ιωάννινα, Μάιος 2026



**ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ**

**Department of Computer Science & Engineering
University of Ioannina**

Ευχαριστίες

Θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου για την καθοδήγηση, τις εύστοχες παρατηρήσεις και την υπομονή του καθ' όλη τη διάρκεια εκπόνησης αυτής της διπλωματικής εργασίας. Οι συζητήσεις μας βοήθησαν καθοριστικά στο να κατανοήσω τις λεπτομέρειες του αλγορίθμου και να οργανώσω την υλοποίηση.

Ευχαριστώ επίσης τα μέλη της τριμελούς εξεταστικής επιτροπής για τον χρόνο και τα σχόλιά τους, καθώς και τους διδάσκοντες του προπτυχιακού προγράμματος σπουδών για τις γνώσεις που μου προσέφεραν σε όλα τα έτη της φοίτησής μου.

Τέλος, ευχαριστώ την οικογένεια και τους φίλους μου για τη στήριξη όλο αυτό το διάστημα.

Περίληψη

Η παρούσα διπλωματική εργασία μελετά τον αλγόριθμο του James B. Orlin για την επίλυση του προβλήματος ροής ελαχίστου κόστους (minimum cost flow) σε πολυωνυμικό και ισχυρά πολυωνυμικό χρόνο, όπως αυτός παρουσιάστηκε στην εργασία του 1993 στο περιοδικό Operations Research. Η εργασία συνοδεύεται από υλοποίηση δύο εκδοχών του αλγορίθμου στη γλώσσα C. Η πρώτη εκδοχή υλοποιεί την τεχνική κλιμάκωσης δεξιού μέλους (RHS-scaling) των Edmonds και Karp, όπως περιγράφεται στην Ενότητα 3 του άρθρου. Η δεύτερη εκδοχή υλοποιεί τον πλήρη ισχυρά πολυωνυμικό αλγόριθμο της Ενότητας 4, ο οποίος προσθέτει στη βασική ιδέα την τεχνική της σύμπτυξης (contraction) ισχυρά εφικτών ακμών, τη χαλαρωμένη επιλογή πηγής και καταβόθρας με παράμετρο α , και το τελικό βήμα ανάκτησης βέλτιστης ροής μέσω υπολογισμού μέγιστης ροής στο υπο-δίκτυο μηδενικού μειωμένου κόστους.

Περιγράφεται λεπτομερώς η αρχιτεκτονική και η λογική κάθε υλοποίησης, ενώ παρέχεται και μια γεννήτρια τυχαίων ισορροπημένων δικτύων ροής σε Python, η οποία χρησιμοποιεί τη βιβλιοθήκη NetworkX και επιτρέπει τον συστηματικό έλεγχο ορθότητας των αλγορίθμων. Τέλος, παρουσιάζεται πειραματική αξιολόγηση των δύο εκδοχών και σύγκρισή τους με τη συνάρτηση υπολογισμού ροής ελαχίστου κόστους της NetworkX, η οποία χρησιμοποιείται ως αναφορά ορθότητας.

Λέξεις κλειδιά: ροή ελαχίστου κόστους, αλγόριθμος Orlin, κλιμάκωση, RHS-scaling, ισχυρά πολυωνυμικός αλγόριθμος, σύμπτυξη κόμβων, συντομότερα μονοπάτια, Dijkstra, δίκτυα.

Abstract

This thesis studies James B. Orlin's algorithm for solving the minimum cost flow problem in polynomial and strongly polynomial time, as presented in his 1993 Operations Research paper. The work is accompanied by an implementation of two versions of the algorithm in the C language. The first version implements the right-hand side scaling (RHS-scaling) technique of Edmonds and Karp, as described in Section 3 of the paper. The second version implements the full strongly polynomial algorithm of Section 4, which augments the basic idea with the contraction of strongly feasible arcs, a relaxed choice of source and sink controlled by a parameter α , and a final step that recovers the optimal flow by computing a maximum flow on the subnetwork of arcs with zero reduced cost.

The architecture and logic of each implementation are described in detail, and a Python generator of random balanced flow networks is also provided; it uses the NetworkX library and enables systematic correctness checks of the algorithms. Finally, the two versions are evaluated experimentally and compared against the min cost flow routine of NetworkX, which is used as a correctness baseline.

Keywords: minimum cost flow, Orlin's algorithm, scaling, RHS-scaling, strongly polynomial algorithm, node contraction, shortest paths, Dijkstra, networks.

Περιεχόμενα

Ευχαριστίες	i
Περίληψη	ii
Abstract	iii
Κεφάλαιο 1. Εισαγωγή	1
1.1 Εισαγωγή	1
1.2 Οργάνωση της διπλωματικής εργασίας	1
1.3 Ιστορική αναδρομή	2

1.4	Εφαρμογές του προβλήματος ροής ελαχίστου κόστους	3
1.5	Θεωρητική Θεμελίωση	3
1.6	Εισαγωγή στο πρόβλημα ροής ελαχίστου κόστους	5
1.7	Συνθήκες βελτιστότητας και δυϊκό πρόβλημα	5
1.8	Επισκόπηση αλγορίθμων για το πρόβλημα ροής ελαχίστου κόστους	6
1.9	Πολυπλοκότητα αλγορίθμων	6
1.10	Προγραμματιστικό περιβάλλον και εργαλεία	7
Κεφάλαιο 2.	Ο αλγόριθμος του Orlin (1993)	8
2.1	Συμβολισμός και ορισμοί	8
2.2	Ψευδοροή και υπολειπόμενο δίκτυο	8
2.3	Δυναμικά κόμβων και μειωμένα κόστη	9
2.4	Η τεχνική κλιμάκωσης των Edmonds-Karp (RHS-Scaling)	9
2.4.1	Φάσεις κλιμάκωσης	9
2.4.2	Επαυξήσεις κατά μήκος συντομότερων μονοπατιών	10
2.4.3	Ορθότητα και πολυπλοκότητα	10
2.5	Από τον RHS-Scaling στον ισχυρά πολυωνυμικό αλγόριθμο	11
2.5.1	Η έννοια της σύμπτυξης	11
2.5.2	Ισχυρά εφικτές ακμές	11
2.5.3	Ένα παθολογικό παράδειγμα	11
2.5.4	Τροποποίηση των κανόνων επαύξησης	12
2.6	Ο ισχυρά πολυωνυμικός αλγόριθμος	12
2.6.1	Περιγραφή αλγορίθμου	12
2.6.2	Ορθότητα και ανάλυση πολυπλοκότητας	13
2.6.3	Αναίρεση συμπτύξεων και εξαγωγή βέλτιστης ροής	14
2.7	Επέκταση σε δίκτυα με χωρητικότητες	14
Κεφάλαιο 3.	Υλοποίηση των αλγορίθμων	15
3.1	Μορφή αρχείου εισόδου και αναπαράσταση δικτύου	15
3.2	Γεννήτρια δικτύων δοκιμών	16
3.3	Υλοποίηση του RHS-Scaling (Section 3)	17
3.3.1	Δομές δεδομένων	17
3.3.2	Αρχικοποίηση και υπολογισμός του Δ	18
3.3.3	Υπολογισμός μειωμένων κοστών	18
3.3.4	Υπολογισμός συντομότερων μονοπατιών (Dijkstra)	18
3.3.5	Ενημέρωση δυναμικών κόμβων	19
3.3.6	Επαύξηση ροής	19
3.3.7	Κύριος βρόχος αλγορίθμου	19
3.4	Υλοποίηση του ισχυρά πολυωνυμικού αλγορίθμου (Section 4)	20
3.4.1	Επέκταση δομών δεδομένων	20
3.4.2	Διαδικασία σύμπτυξης κόμβων	21
3.4.3	Αναγνώριση και σύμπτυξη ισχυρά εφικτών ακμών	22
3.4.4	Επιλογή πηγής και καταβόθρας με κατώφλι α	22

3.4.5	Πλήρης αναγέννηση	23
3.4.6	Αναίρεση συμπτύξεων	23
3.4.7	Υπολογισμός βέλτιστης ροής μέσω μέγιστης ροής	24
3.5	Παράδειγμα εκτέλεσης βήμα προς βήμα	24
Κεφάλαιο 4.	Πειραματική Αξιολόγηση	26
4.1	Μεθοδολογία πειραμάτων	26
4.2	Έλεγχος ορθότητας με NetworkX	26
4.3	Πειραματικά αποτελέσματα	27
4.3.1	Επίδραση του μεγέθους δικτύου	27
4.3.2	Επίδραση του αριθμού πηγών και καταβοθρών	30
4.3.3	Επίδραση του εύρους τιμών προσφοράς/ζήτησης	31
4.4	Σύνοψη πειραματικών αποτελεσμάτων	33
Κεφάλαιο 5.	Συμπεράσματα	34
5.1	Συμπεράσματα	34
5.2	Μελλοντικές επεκτάσεις	34
Βιβλιογραφία		36
Κεφάλαιο Α'.	Μετάφραση ξένων όρων	37

Κεφάλαιο 1. Εισαγωγή

1.1 Εισαγωγή

Το πρόβλημα ροής ελαχίστου κόστους αποτελεί ένα από τα πιο θεμελιώδη προβλήματα στη θεωρία των δικτύων ροής. Δίνεται ένα κατευθυνόμενο δίκτυο στο οποίο κάθε κόμβος φέρει μια τιμή προσφοράς ή ζήτησης, κάθε ακμή φέρει ένα κόστος ανά μονάδα ροής και προαιρετικά μια άνω χωρητικότητα, και ζητείται να βρεθεί μια εφικτή ροή που ικανοποιεί τους περιορισμούς ισορροπίας μάζας σε κάθε κόμβο και ελαχιστοποιεί το συνολικό κόστος.

Το πρόβλημα αυτό έχει μελετηθεί εκτενώς λόγω της γενικότητάς του. Πολλά γνωστά προβλήματα συνδυαστικής βελτιστοποίησης, όπως το συντομότερο μονοπάτι, η μέγιστη ροή, η αντιστοίχιση σε διμερή γραφήματα και η κατανομή πόρων, αποτελούν ειδικές περιπτώσεις του. Για αυτόν τον λόγο, η αποδοτική επίλυσή του απασχόλησε τους ερευνητές για δεκαετίες και οδήγησε σε μια πλούσια αλγοριθμική θεωρία.

Ένα από τα πιο σημαντικά επιτεύγματα στην ιστορία του προβλήματος είναι ο ισχυρά πολυωνυμικός αλγόριθμος του James B. Orlin, ο οποίος δημοσιεύτηκε το 1993 στο περιοδικό *Operations Research*. Ο αλγόριθμος βασίζεται σε μια βελτιωμένη εκδοχή της τεχνικής κλιμάκωσης δεξιού μέλους (*right-hand side scaling*) των Edmonds και Karp, εμπλουτισμένη με την τεχνική της σύμπτυξης κόμβων. Επιλύει το μη χωρητικό πρόβλημα σε χρόνο ανάλογο με ένα πλήθος συντομότερων μονοπατιών που εξαρτάται μόνο από τον αριθμό κόμβων και ακμών, χωρίς εξάρτηση από τις αριθμητικές τιμές των κοστών ή των χωρητικοτήτων.

Η παρούσα διπλωματική εργασία μελετά τον αλγόριθμο αυτόν και τον υλοποιεί σε δύο διακριτές εκδοχές σε γλώσσα C. Η πρώτη εκδοχή υλοποιεί την απλή τεχνική κλιμάκωσης, η οποία επιστρέφει πολυωνυμικό (αλλά όχι ισχυρά πολυωνυμικό) χρόνο. Η δεύτερη εκδοχή υλοποιεί ολόκληρο τον ισχυρά πολυωνυμικό αλγόριθμο, συμπεριλαμβανομένης της σύμπτυξης ισχυρά εφικτών ακμών, της αναίρεσης συμπτύξεων στο τέλος και του υπολογισμού βέλτιστης ροής μέσω μέγιστης ροής στο υπο-δίκτυο μηδενικού μειωμένου κόστους.

1.2 Οργάνωση της διπλωματικής εργασίας

Η εργασία οργανώνεται σε πέντε κεφάλαια, όπως περιγράφεται παρακάτω.

Στο Κεφάλαιο 1 παρουσιάζεται το αντικείμενο της εργασίας, η ιστορική εξέλιξη του προβλήματος, οι κυριότερες εφαρμογές του, οι βασικές θεωρητικές έννοιες που θα χρησιμοποιηθούν στη συνέχεια, καθώς και το προγραμματιστικό περιβάλλον της υλοποίησης.

Στο Κεφάλαιο 2 αναλύεται θεωρητικά ο αλγόριθμος του Orlin. Περιγράφονται οι βασικοί ορισμοί, η έννοια της ψευδοροής και του υπολειπόμενου δικτύου, οι συνθήκες βελτιστότητας, η τεχνική κλιμάκωσης δεξιού μέλους και τα επιπλέον βήματα που απαιτούνται για την επέκταση σε ισχυρά πολυωνυμικό χρόνο.

Στο Κεφάλαιο 3 παρουσιάζεται αναλυτικά η υλοποίηση και των δύο εκδοχών του αλγορίθμου σε γλώσσα C. Αναλύονται οι δομές δεδομένων, οι βασικές συναρτήσεις και οι λογικές επιλογές που έγιναν, καθώς και η γεννήτρια τυχαίων δικτύων σε Python που χρησιμοποιήθηκε για την παραγωγή περιπτώσεων δοκιμής.

Στο Κεφάλαιο 4 παρουσιάζεται η πειραματική αξιολόγηση των δύο υλοποιήσεων. Συγκρίνονται ως προς την ορθότητα (χρησιμοποιώντας τη NetworkX ως αναφορά) και ως προς την απόδοση σε διάφορα μεγέθη και πυκνότητες δικτύων.

Στο Κεφάλαιο 5 συνοψίζονται τα κύρια συμπεράσματα και προτείνονται πιθανές επεκτάσεις της εργασίας.

1.3 Ιστορική αναδρομή

Το πρόβλημα της ροής ελαχίστου κόστους έχει μακρά ιστορία, η οποία ξεκίνησε τη δεκαετία του 1960 με κλασικούς αλγορίθμους όπως η μέθοδος των Busacker και Gowen, οι οποίοι επαύξαναν τη ροή κατά μήκος συντομότερων μονοπατιών ως προς το κόστος, και η μέθοδος ακύρωσης κύκλων αρνητικού κόστους του Klein. Οι πρώτες αυτές προσεγγίσεις, αν και ορθές, δεν ήταν πολυωνυμικές στον αυστηρό αριθμητικό περιορισμό του χρόνου.

Το καθοριστικό πρώτο βήμα στην κατεύθυνση πολυωνυμικών αλγορίθμων έγινε το 1972 από τους Jack Edmonds και Richard Karp, οι οποίοι εισήγαγαν την τεχνική κλιμάκωσης δεξιού μέλους (RHS-scaling). Η τεχνική τους ανάγει το πρόβλημα σε μια ακολουθία λογαριθμικά πολλών φάσεων, σε καθεμία από τις οποίες επιλύονται λίγα προβλήματα συντομότερου μονοπατιού.

Το πιο σημαντικό θεωρητικό ερώτημα στον χώρο παρέμενε για χρόνια ανοιχτό: υπάρχει αλγόριθμος του οποίου ο χρόνος εκτέλεσης εξαρτάται μόνο από τις διαστάσεις του δικτύου και όχι από τις αριθμητικές τιμές των δεδομένων; Την καταφατική απάντηση έδωσε πρώτη η Έβα Tardos το 1985 με έναν ισχυρά πολυωνυμικό αλγόριθμο πολυπλοκότητας $O(m^4)$. Άλλοι ισχυρά πολυωνυμικοί αλγόριθμοι αναπτύχθηκαν από τους Orlin (1984), Fujishige (1986), Galil και Tardos (1986), καθώς και από τους Goldberg και Tarjan (1987, 1988).

Ο αλγόριθμος του Orlin του 1993, τον οποίο μελετά αυτή η εργασία, βελτίωσε τον καλύτερο τότε χρόνο των Galil και Tardos κατά έναν παράγοντα n^2/m και έδωσε έναν

αλγόριθμο που επιλύει το μη χωρητικό πρόβλημα ως ακολουθία $O(n \log n)$ συντομότερων μονοπατιών. Με χρήση της υλοποίησης Fibonacci heap του Dijkstra, ο συνολικός χρόνος είναι $O(n \log n (m + n \log n))$.

1.4 Εφαρμογές του προβλήματος ροής ελαχίστου κόστους

Το πρόβλημα της ροής ελαχίστου κόστους εμφανίζεται σε πλήθος πρακτικών εφαρμογών. Ενδεικτικά, μερικές κατηγορίες εφαρμογών παρουσιάζονται παρακάτω.

Μεταφορές και logistics. Η κλασική περίπτωση είναι το πρόβλημα της μεταφοράς αγαθών από αποθήκες προς καταναλωτές με ελάχιστο κόστος μεταφοράς. Οι κόμβοι-αποθήκες έχουν θετική προσφορά, οι κόμβοι-αγορές έχουν ζήτηση, και τα κόστη των ακμών αντιπροσωπεύουν το κόστος αποστολής ανά μονάδα προϊόντος.

Δίκτυα τηλεπικοινωνιών. Η διανομή κίνησης μεταξύ κεντρικών κόμβων ενός δικτύου, όπου κάθε σύνδεση έχει χωρητικότητα και κόστος, μπορεί να μοντελοποιηθεί ως πρόβλημα ροής ελαχίστου κόστους. Το ίδιο ισχύει για τη δρομολόγηση πακέτων και τη διαχείριση εύρους ζώνης.

Προγραμματισμός παραγωγής. Προβλήματα επιλογής παραγωγικών γραμμών, αποθήκευσης προϊόντων στον χρόνο και ικανοποίησης περιοδικής ζήτησης αναδιαρθρώνονται συχνά ως ροές σε δίκτυα με δομή διαμερισμένη κατά χρονικά βήματα.

Αντιστοίχιση σε διμερή γραφήματα. Το πρόβλημα της τέλει αντιστοίχισης ελαχίστου κόστους σε διμερές γράφημα είναι ειδική περίπτωση του προβλήματος ροής ελαχίστου κόστους και επιλύεται απευθείας από τους ίδιους αλγόριθμους.

Υπολογιστική όραση. Ορισμένα προβλήματα τμηματοποίησης εικόνας και αντιστοίχισης χαρακτηριστικών μεταξύ εικόνων αναδιαρθρώνονται ως προβλήματα ελαχίστου κόστους ροής.

1.5 Θεωρητική Θεμελίωση

Αυτή η ενότητα συνοψίζει τις θεμελιώδεις έννοιες της θεωρίας γραφημάτων και δικτύων ροής που χρησιμοποιούνται στη συνέχεια. Για περισσότερες λεπτομέρειες παραπέμπουμε στο εγχειρίδιο των Ahuja, Magnanti και Orlin.

Ορισμός 1.1 (Κατευθυνόμενο γράφημα). *Κατευθυνόμενο γράφημα είναι ένα ζεύγος $G = (V, E)$, όπου V είναι ένα πεπερασμένο σύνολο κόμβων και $E \subseteq V \times V$ είναι ένα σύνολο διατεταγμένων ζευγών κόμβων, που ονομάζονται ακμές. Για κάθε ακμή (u, v) , ο u είναι η ουρά και ο v η κεφαλή της ακμής.*

Ορισμός 1.2 (Δίκτυο ροής). *Δίκτυο ροής είναι ένα κατευθυνόμενο γράφημα $G = (V, E)$ εφοδιασμένο με:*

- συνάρτηση κόστους $c : E \rightarrow \mathbb{Z}$ που αντιστοιχίζει σε κάθε ακμή ένα ακέραιο κό-

στος ανά μονάδα ροής·

- συνάρτηση προσφοράς/ζήτησης $b : V \rightarrow \mathbb{Z}$, όπου $b(v) > 0$ σημαίνει ότι ο κόμβος v προσφέρει $b(v)$ μονάδες, $b(v) < 0$ σημαίνει ότι ο κόμβος απορροφά $|b(v)|$ μονάδες, και $b(v) = 0$ αντιστοιχεί σε κόμβο διαμεταγωγής·
- προαιρετικά, συνάρτηση χωρητικότητας $u : E \rightarrow \mathbb{Z}_+ \cup \{\infty\}$ που θέτει άνω όριο ροής.

Ορισμός 1.3 (Ροή). Ροή είναι συνάρτηση $x : E \rightarrow \mathbb{R}_+$ που σε κάθε ακμή (i, j) αποδίδει μια μη αρνητική τιμή x_{ij} . Η ροή ονομάζεται εφικτή αν ικανοποιεί τους περιορισμούς ισορροπίας μάζας σε κάθε κόμβο:

$$\sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = b(i), \quad \forall i \in V.$$

Ορισμός 1.4 (Ψευδοροή). Ψευδοροή είναι μια συνάρτηση $x : E \rightarrow \mathbb{R}_+$ που ικανοποιεί μόνο τους περιορισμούς μη αρνητικότητας, αλλά επιτρέπεται να παραβιάζει τους περιορισμούς ισορροπίας μάζας. Για δοθείσα ψευδοροή x , το πλεόνασμα ή imbalance του κόμβου i ορίζεται ως

$$e(i) = b(i) + \sum_{j:(j,i) \in E} x_{ji} - \sum_{j:(i,j) \in E} x_{ij}.$$

Αν $e(i) > 0$, ο i λέγεται πηγή· αν $e(i) < 0$, λέγεται καταβόθρα· αν $e(i) = 0$, λέγεται ισορροπημένος.

Ορισμός 1.5 (Υπολειπόμενο δίκτυο). Για δοθείσα ψευδοροή x , το υπολειπόμενο δίκτυο $G(x)$ προκύπτει από το G με τον εξής τρόπο: κάθε ακμή $(i, j) \in E$ του αρχικού αντικαθίσταται από δύο ακμές, την προσθετική (i, j) με κόστος c_{ij} και άπειρη υπολειπόμενη χωρητικότητα (στην μη χωρητική περίπτωση) και την αφαιρετική (j, i) με κόστος $-c_{ij}$ και υπολειπόμενη χωρητικότητα ίση με την τρέχουσα ροή x_{ij} . Στο υπολειπόμενο δίκτυο διατηρούμε μόνο ακμές με θετική υπολειπόμενη χωρητικότητα.

Ορισμός 1.6 (Δυναμικά κόμβων και μειωμένο κόστος). Σε κάθε κόμβο i αντιστοιχούμε μια δυική μεταβλητή $\pi(i)$ που ονομάζεται δυναμικό. Το μειωμένο κόστος μιας ακμής (i, j) ως προς τα δυναμικά π ορίζεται ως

$$c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j).$$

1.6 Εισαγωγή στο πρόβλημα ροής ελαχίστου κόστους

Το μη χωρητικό πρόβλημα ροής ελαχίστου κόστους διατυπώνεται ως το παρακάτω πρόβλημα γραμμικού προγραμματισμού:

$$\begin{aligned} & \text{ελαχιστοποίηση} && \sum_{(i,j) \in E} c_{ij} x_{ij} \\ & \text{υπό} && \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = b(i), \quad \forall i \in V \\ & && x_{ij} \geq 0, \quad \forall (i,j) \in E. \end{aligned}$$

Θα υποθέσουμε στη συνέχεια, ακολουθώντας τη σύμβαση του Orlin, ότι το δίκτυο είναι ισχυρά συνδεδεμένο και ότι όλα τα κόστη είναι μη αρνητικά. Οι δύο αυτές παραδοχές δεν αποτελούν περιορισμό, καθώς μπορούν να ικανοποιηθούν με κλασικούς μετασχηματισμούς (προσθήκη τεχνητού κόμβου με τεχνητές ακμές μεγάλου κόστους για ισχυρή συνδεδεμένη, αλλαγή κόστους μέσω δυναμικών για μη αρνητικότητα).

Το πρόβλημα διαθέτει μια κλασική δυική διατύπωση. Σε κάθε περιορισμό ισορροπίας μάζας αντιστοιχούμε μια δυική μεταβλητή $\pi(i)$. Τότε το δυικό πρόβλημα έχει τη μορφή:

$$\begin{aligned} & \text{μεγιστοποίηση} && \sum_{i \in V} b(i) \pi(i) \\ & \text{υπό} && \pi(i) - \pi(j) \leq c_{ij}, \quad \forall (i,j) \in E. \end{aligned}$$

1.7 Συνθήκες βελτιστότητας και δυικό πρόβλημα

Οι συνθήκες συμπληρωματικής χαλαρότητας για το πρωτεύον-δυικό ζεύγος γίνονται εξαιρετικά διαφωτιστικές. Χρησιμοποιώντας τα μειωμένα κόστη $c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j)$, οι συνθήκες βελτιστότητας εκφράζονται στο υπολειπόμενο δίκτυο ως εξής:

Μια ροή x είναι βέλτιστη αν και μόνο αν υπάρχει διάνυσμα δυναμικών π τέτοιο ώστε όλες οι ακμές του υπολειπόμενου δικτύου $G(x)$ να έχουν μη αρνητικό μειωμένο κόστος, δηλαδή

$$c_{ij}^\pi \geq 0, \quad \forall (i,j) \in E(G(x)).$$

Η ισοδυναμία αυτή έχει θεμελιώδεις συνέπειες για τον σχεδιασμό αλγορίθμων. Αν μια τρέχουσα ψευδοροή x και τρέχοντα δυναμικά π ικανοποιούν την παραπάνω συνθήκη, τότε κάθε βελτίωση της ροής μπορεί να γίνει μόνο κατά μήκος μονοπατιών μηδενικού μειωμένου κόστους, ή ακόμα πιο αποδοτικά, κατά μήκος συντομότερων μονοπατιών ως προς μειωμένα κόστη από μια πηγή σε μια καταβόθρα. Ένα βασικό λήμμα (Λήμμα 3 του άρθρου) εγγυάται ότι, αν αντικαταστήσουμε τα δυναμικά ως $\pi := \pi - d$, όπου d είναι οι αποστάσεις συντομότερου μονοπατιού ως προς τα μειωμένα κόστη, τα νέα δυναμικά διατηρούν τη συνθήκη βελτιστότητας και επιπλέον οι ακμές του συντομότερου μονοπατιού αποκτούν μηδενικό μειωμένο κόστος, οπότε η επαύξηση κατά μήκος του

μονοπατιού είναι ασφαλής.

1.8 Επισκόπηση αλγορίθμων για το πρόβλημα ροής ελαχίστου κόστους

Η έρευνα στον χώρο των αλγορίθμων για το πρόβλημα ροής ελαχίστου κόστους χωρίζεται σε δύο μεγάλες κατηγορίες. Η πρώτη αφορά πολυωνυμικούς αλγορίθμους που εξαρτώνται από τις αριθμητικές τιμές του δικτύου μέσω λογαριθμικών όρων (π.χ. $\log C$ για ακέραια κόστη με μέγιστη απόλυτη τιμή C). Η δεύτερη αφορά ισχυρά πολυωνυμικούς αλγορίθμους, των οποίων ο χρόνος εκτέλεσης εκφράζεται αποκλειστικά μέσω του n και του m .

Στην πρώτη κατηγορία ανήκουν οι αλγόριθμοι κλιμάκωσης των Edmonds και Karp (1972) και του Röck (1980), οι αλγόριθμοι διπλής κλιμάκωσης των Ahuja, Goldberg, Orlin και Tarjan (1988), καθώς και ο αλγόριθμος διαδοχικής προσέγγισης των Goldberg και Tarjan (1987). Στη δεύτερη κατηγορία ανήκουν οι αλγόριθμοι της Tardos (1985), του Fujishige (1986), των Galil και Tardos (1986), καθώς και ο αλγόριθμος του Orlin (1993).

Για τη σύγκριση των αλγορίθμων αυτών παραπέμπουμε στον πίνακα της Εικόνας 1 του άρθρου του Orlin, ο οποίος συνοψίζει τη ροή εξελίξεων. Στην παρούσα εργασία μας ενδιαφέρει κυρίως η σύγκριση του βασικού αλγορίθμου κλιμάκωσης των Edmonds-Karp (την εκδοχή που υλοποιεί η Ενότητα 3 του άρθρου) με τον ισχυρά πολυωνυμικό αλγόριθμο του Orlin (Ενότητα 4).

1.9 Πολυπλοκότητα αλγορίθμων

Η χρονική πολυπλοκότητα ενός αλγορίθμου εκφράζεται συνήθως ως συνάρτηση του μεγέθους της εισόδου, όπου ως μέγεθος εννοούμε συνήθως το πλήθος κόμβων n και ακμών m ενός δικτύου. Στον χώρο των αλγορίθμων δικτύων ροής, είναι κλασικό να εκφράζουμε τους χρόνους ως γινόμενα του τύπου [αριθμός επαναλήψεων] $\times$ [χρόνος επαναληπτικού βήματος].

Ο αλγόριθμος της Ενότητας 3 του Orlin επιλύει το μη χωρητικό πρόβλημα σε $O(n \log U)$ συντομότερα μονοπάτια, όπου U η μέγιστη απόλυτη τιμή της προσφοράς/ζήτησης. Ο αλγόριθμος της Ενότητας 4 τα επιλύει σε $O(n \log n)$ συντομότερα μονοπάτια, χωρίς εξάρτηση από τις αριθμητικές τιμές.

Στην υλοποίησή μας χρησιμοποιούμε την απλή υλοποίηση του Dijkstra με αναζήτηση γραμμικού χρόνου της ελάχιστης απόστασης, οπότε κάθε κλήση συντομότερου μονοπατιού τρέχει σε $O(n^2 + m)$ χρόνο. Αυτό απλοποιεί τον κώδικα αλλά δίνει ένα χειρότερο άνω όριο από το θεωρητικά βέλτιστο που πετυχαίνεται με Fibonacci heaps. Επιλέξαμε αυτήν την προσέγγιση γιατί επικεντρωνόμαστε στη σωστή ενσάρκωση της αλγοριθμικής δομής του Orlin, όχι στη βέλτιστη εκμετάλλευση δομών δεδομένων για τον Dijkstra.

1.10 Προγραμματιστικό περιβάλλον και εργαλεία

Η υλοποίηση του αλγορίθμου έγινε σε γλώσσα C. Η επιλογή της C έγινε κυρίως για λόγους απόδοσης και διαφάνειας στη διαχείριση μνήμης: όλες οι δομές δεδομένων δηλώνονται ρητά, όλες οι εκχωρήσεις γίνονται χειροκίνητα και η δομή του κώδικα αντιστοιχεί πιστά στα βήματα του αλγορίθμου όπως περιγράφονται στο άρθρο.

Ο κώδικας μεταγλωττίζεται με τον μεταγλωττιστή gcc. Το Makefile που συνοδεύει την εργασία ορίζει δύο στόχους:

- `mcf_section3`: υλοποιεί την τεχνική κλιμάκωσης των Edmonds-Karp (Ενότητα 3 του άρθρου).
- `mcf_section4`: υλοποιεί τον ισχυρά πολυωνυμικό αλγόριθμο (Ενότητα 4 του άρθρου).

Οι σημαίες μεταγλώττισης που χρησιμοποιούνται είναι `-Wall -Wextra -g`, οι οποίες ενεργοποιούν όλες τις προειδοποιήσεις και συμπεριλαμβάνουν πληροφορίες εκσφαλμάτωσης.

Για τη δημιουργία τυχαίων δικτύων δοκιμής χρησιμοποιείται ένα σενάριο Python (`generator.py`) που αξιοποιεί τη βιβλιοθήκη NetworkX. Η NetworkX προσφέρει εύκολη κατασκευή κατευθυνόμενων γραφημάτων, ελέγχους ισχυρής συνδεδεμένης και, σημαντικό για την εργασία μας, ενσωματωμένη υλοποίηση επιλύτη ροής ελαχίστου κόστους (συνάρτηση `min_cost_flow_cost`), την οποία χρησιμοποιούμε ως αναφορά ορθότητας. Η γεννήτρια υποστηρίζει παράμετρο `seed` για αναπαραγωγικότητα των πειραμάτων.

Η μορφή αρχείου εισόδου που υποστηρίζουν και οι δύο υλοποιήσεις είναι η εξής: η πρώτη γραμμή περιέχει δύο ακεραίους, n και m , τον αριθμό κόμβων και ακμών αντίστοιχα. Η δεύτερη γραμμή περιέχει n ακεραίους χωρισμένους με κενό, τις τιμές $b(i)$ για κάθε κόμβο. Ακολουθούν m γραμμές, καθεμία στη μορφή `from to cost`, που περιγράφουν τις ακμές.

Κεφάλαιο 2. Ο αλγόριθμος του Orlin (1993)

2.1 Συμβολισμός και ορισμοί

Θεωρούμε κατευθυνόμενο δίκτυο $G = (V, E)$ με $|V| = n$ και $|E| = m$. Σε κάθε ακμή $(i, j) \in E$ αντιστοιχίζεται ακέραιο κόστος $c_{ij} \geq 0$. Σε κάθε κόμβο i αντιστοιχίζεται ακέραιος αριθμός $b(i)$, που δηλώνει την προσφορά (αν θετικός) ή τη ζήτηση (αν αρνητικός) του κόμβου. Θέτουμε $U = \max_i |b(i)|$.

Η ροή σημειώνεται $x = (x_{ij})$ και αποτελεί διάνυσμα επί των ακμών. Δεδομένης ροής x , το πλεόνασμα του κόμβου i είναι

$$e(i) = b(i) + \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}.$$

Σημειώνουμε ως

$$S(\Delta) = \{i \in V : e(i) \geq \Delta\}, \quad T(\Delta) = \{i \in V : e(i) \leq -\Delta\}$$

τα σύνολα κόμβων με πλεόνασμα ή έλλειμμα τουλάχιστον Δ , αντίστοιχα.

Σε κάθε κόμβο αντιστοιχίζεται δυναμικό $\pi(i) \in \mathbb{R}$. Το μειωμένο κόστος μιας ακμής (i, j) είναι $c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j)$. Το υπολειπόμενο δίκτυο $G(x)$ περιέχει, για κάθε ακμή $(i, j) \in E$, την προσθετική ακμή (i, j) με κόστος c_{ij} και άπειρη υπολειπόμενη χωρητικότητα, και την αφαιρετική ακμή (j, i) με κόστος $-c_{ij}$ και υπολειπόμενη χωρητικότητα x_{ij} (η οποία υπάρχει μόνο αν $x_{ij} > 0$).

2.2 Ψευδοροή και υπολειπόμενο δίκτυο

Η έννοια της ψευδοροής είναι κεντρική στον αλγόριθμο του Orlin. Αντί να διατηρούμε σε κάθε βήμα μια εφικτή ροή και να την βελτιώνουμε, διατηρούμε μια ψευδοροή (η οποία ενδέχεται να παραβιάζει την ισορροπία σε ορισμένους κόμβους) και προσπαθούμε να την κάνουμε εφικτή. Τα πλεονάσματα $e(i)$ παρακολουθούν αυτή τη διαφορά από την ισορροπία.

Η κεντρική λειτουργία είναι η επαύξηση κατά μήκος μονοπατιού από πηγή προς καταβόθρα. Αν ο s είναι κόμβος με $e(s) \geq \Delta$ και ο t είναι κόμβος με $e(t) \leq -\Delta$, και υπάρχει μονοπάτι P στο $G(x)$ από τον s στον t , τότε μπορούμε να στείλουμε Δ μονάδες ροής κατά μήκος του P . Αυτό μειώνει το $e(s)$ κατά Δ και αυξάνει το $e(t)$ κατά Δ , ενώ ενημε-

ρώνει τις υπολειπόμενες χωρητικότητες του μονοπατιού.

Για να αποφύγουμε την αναζήτηση κύκλων αρνητικού μειωμένου κόστους, επιλέγουμε πάντοτε το συντομότερο μονοπάτι από τον s στον t ως προς τα μειωμένα κόστη. Το παρακάτω λήμμα (Λήμμα 3 στο άρθρο του Orlin) διασφαλίζει ότι αυτή η επιλογή διατηρεί τη συνθήκη βελτιστότητας.

2.3 Δυναμικά κόμβων και μειωμένα κόστη

Λήμμα 2.1 (Ενημέρωση δυναμικών). Έστω ψευδοροή x που ικανοποιεί τη συνθήκη βελτιστότητας ως προς δυναμικά π , και έστω $d(i)$ η απόσταση συντομότερου μονοπατιού από τον κόμβο-πηγή s σε κάθε κόμβο i στο $G(x)$, χρησιμοποιώντας μειωμένα κόστη c^π ως μήκη ακμών. Θέτουμε $\pi' = \pi - d$. Τότε:

1. Η ψευδοροή x ικανοποιεί τη συνθήκη βελτιστότητας και ως προς τα π' .
2. Για κάθε ακμή (i, j) επί ενός συντομότερου μονοπατιού από τον s , το μειωμένο κόστος ως προς τα π' είναι μηδέν.

Η ιδιότητα αυτή είναι καθοριστική: ύστερα από επαύξηση κατά μήκος του συντομότερου μονοπατιού, οι νέες αφαιρετικές ακμές που εμφανίζονται στο υπολειπόμενο δίκτυο έχουν επίσης μηδενικό μειωμένο κόστος (αρνητικό του μηδενικού, δηλαδή πάλι μηδέν), οπότε η συνθήκη βελτιστότητας συνεχίζει να ισχύει.

2.4 Η τεχνική κλιμάκωσης των Edmonds-Karp (RHS-Scaling)

2.4.1 Φάσεις κλιμάκωσης

Η τεχνική της κλιμάκωσης δεξιού μέλους επιλύει το πρόβλημα ως μια ακολουθία φάσεων, καθεμία από τις οποίες αντιστοιχεί σε μια τιμή κλίμακας Δ . Αρχικά θέτουμε $\Delta = 2^{\lceil \log U \rceil}$. Σε κάθε φάση, η τιμή Δ μειώνεται στο μισό. Σε κάθε φάση εξετάζουμε τα σύνολα $S(\Delta)$ και $T(\Delta)$ και, όσο υπάρχει κόμβος σε κάθε ένα, εκτελούμε επαύξηση κατά μήκος συντομότερου μονοπατιού που στέλνει Δ μονάδες.

Ένα κλειδί είναι ότι κάθε φάση απαιτεί μόνο λίγες επαυξήσεις. Στην αρχή κάθε φάσης ισχύει ότι $S(2\Delta) = \emptyset$ ή $T(2\Delta) = \emptyset$. Στην πρώτη περίπτωση, κάθε επαύξηση αφαιρεί έναν κόμβο από το $S(\Delta)$ · στη δεύτερη, αφαιρεί έναν κόμβο από το $T(\Delta)$. Έτσι το Λήμμα 6 του άρθρου δίνει ότι ο αριθμός επαυξήσεων σε μια φάση δεν ξεπερνά τον αριθμό κόμβων που αναγεννώνται στην αρχή της φάσης. Συνεπώς, το συνολικό πλήθος επαυξήσεων είναι $O(n \log U)$.

2.4.2 Επαυξήσεις κατά μήκος συντομότερων μονοπατιών

Το κρίσιμο λήμμα (Λήμμα 5 του άρθρου) εγγυάται ότι, σε κάθε σημείο του αλγορίθμου, κάθε ροή ή υπολειπόμενη χωρητικότητα είναι ακέραιο πολλαπλάσιο του τρέχοντος Δ . Αυτό εξασφαλίζει ότι μπορούμε πάντοτε να στέλνουμε ολόκληρα Δ μονάδων κατά μήκος οποιουδήποτε μονοπατιού με θετική υπολειπόμενη χωρητικότητα.

Αλγόριθμος 1: RHS-Scaling (Εικόνα 3 του άρθρου του Orlin)

Είσοδος: Δίκτυο $G = (V, E)$ με κόστη c και τιμές $b(\cdot)$

Έξοδος: Βέλτιστη ροή x και δυναμικά π

```

1  $x \leftarrow 0$ ;
2  $\pi \leftarrow 0$ ;
3  $e \leftarrow b$ ;
4  $U \leftarrow \max\{|b(i)| : i \in V\}$ ;
5  $\Delta \leftarrow 2^{\lceil \log U \rceil}$ ;
6 ενώσω υπάρχει  $i$  με  $e(i) \neq 0$  επανάλαβε
    /*  $\Delta$ -φάση κλιμάκωσης */
7    $S(\Delta) \leftarrow \{i : e(i) \geq \Delta\}$ ;
8    $T(\Delta) \leftarrow \{i : e(i) \leq -\Delta\}$ ;
9   ενώσω  $S(\Delta) \neq \emptyset$  και  $T(\Delta) \neq \emptyset$  επανάλαβε
10    επέλεξε  $k \in S(\Delta)$  και  $v \in T(\Delta)$ ;
11    υπολόγισε αποστάσεις  $d(i)$  από τον  $k$  στο  $G(x)$  με μήκη  $c^\pi$ ;
12     $\pi(i) \leftarrow \pi(i) - d(i), \forall i \in V$ ;
13    επαύξησε  $\Delta$  μονάδες κατά μήκος του συντομότερου μονοπατιού  $k \rightarrow v$ ;
14    ενημέρωσε  $x, e, S(\Delta), T(\Delta)$ ;
15   $\Delta \leftarrow \Delta/2$ ;
```

2.4.3 Ορθότητα και πολυπλοκότητα

Η ορθότητα προκύπτει από τον συνδυασμό τριών παρατηρήσεων. Πρώτον, η αρχικοποίηση ($x = 0, \pi = 0$) ικανοποιεί τη συνθήκη βελτιστότητας, καθώς όλες οι ακμές του υπολειπόμενου δικτύου έχουν μη αρνητικό κόστος. Δεύτερον, κάθε επαύξηση με ενημέρωση δυναμικών διατηρεί τη συνθήκη βελτιστότητας. Τρίτον, όταν $\Delta < 1$ και όλοι οι ακέραιοι αρχικοί αριθμοί έχουν εξαντλήσει τις κλίμακες, η ακεραιότητα εξασφαλίζει ότι όλα τα πλεονάσματα έχουν μηδενιστεί, άρα η ψευδοροή είναι πλέον εφικτή ροή.

Η συνολική πολυπλοκότητα του RHS-Scaling για το μη χωρητικό πρόβλημα είναι $O(n \log U)$ συντομότερα μονοπάτια. Αν σημειώσουμε $S(n, m)$ τον χρόνο υπολογισμού ενός συντομότερου μονοπατιού σε γράφημα με n κόμβους και m ακμές με μη αρνητικά κόστη, ο συνολικός χρόνος γίνεται $O(n \log U \cdot S(n, m))$.

2.5 Από τον RHS-Scaling στον ισχυρά πολυωνυμικό αλγόριθμο

Το μεγάλο άλμα του Orlin (1993) είναι η δυνατότητα να αντικατασταθεί ο παράγοντας $\log U$ με παράγοντα $\log n$, κάτι που οδηγεί σε αλγόριθμο του οποίου ο χρόνος δεν εξαρτάται από τις αριθμητικές τιμές της εισόδου.

Για να το πετύχει αυτό, ο αλγόριθμος εισάγει τρεις ιδέες: (α) την έννοια της ισχυρά εφικτής ακμής και τη σύμπτυξη, (β) μια τροποποίηση της επιλογής πηγής και καταβόθρας μέσω μιας παραμέτρου $\alpha \in (1/2, 1)$, και (γ) έναν μηχανισμό πλήρους αναγέννησης που επαναφέρει το Δ στη μέγιστη τρέχουσα τιμή πλεονάσματος όταν οι ροές μηδενίζονται.

2.5.1 Η έννοια της σύμπτυξης

Η βασική διαίσθηση είναι η εξής. Αν σε μια χρονική στιγμή διαπιστώσουμε ότι η ροή μιας ακμής είναι αρκετά μεγάλη ώστε να αποκλειστεί η περίπτωση να μηδενιστεί σε κάποια επόμενη φάση, τότε γνωρίζουμε ότι αυτή η ακμή θα έχει θετική ροή στη βέλτιστη λύση. Από τις συνθήκες συμπληρωματικής χαλαρότητας έπεται ότι το βέλτιστο μειωμένο κόστος αυτής της ακμής είναι μηδέν. Αν μετασχηματίσουμε τα κόστη του δικτύου σε μειωμένα κόστη, η ακμή αποκτά μηδενικό κόστος και τα βέλτιστα δυναμικά των δύο άκρων της γίνονται ίσα.

Αυτή η ισοδυναμία επιτρέπει τη σύμπτυξη των δύο άκρων σε έναν κόμβο: το δίκτυο μικραίνει κατά έναν κόμβο, και το νέο δίκτυο έχει ως λύση ουσιαστικά την ίδια βέλτιστη ροή. Επειδή οι συμπτώξεις είναι το πολύ $n - 1$, η συνολική μείωση μεγέθους οδηγεί σε ανάλογη μείωση στον αριθμό φάσεων.

2.5.2 Ισχυρά εφικτές ακμές

Για να εκφραστεί τυπικά η παραπάνω διαίσθηση, ο Orlin εισάγει στην Ενότητα 4.1 του άρθρου την έννοια της *ισχυρά εφικτής ακμής*: μια ακμή (i, j) ονομάζεται ισχυρά εφικτή στη Δ -φάση αν η ροή της ξεπερνά το $2n\Delta$. Στον ίδιο τον αλγόριθμο, ωστόσο, η σύμπτυξη ενεργοποιείται μόνο όταν η ροή φτάσει το αυστηρότερο όριο $3n\Delta$, ώστε να συνυπολογιστούν και οι έως $n - 1$ επιπλέον επαυξήσεις που προκαλούν οι ίδιες οι συμπτώξεις. Όταν μια ακμή ικανοποιήσει αυτό το κατώφλι, συμπτύσσονται τα άκρα της.

Το Λήμμα 11 του άρθρου εγγυάται ότι, αν $x_{ij} \geq 3n\Delta$ σε μια Δ -φάση, τότε $x_{ij} > 0$ σε όλες τις μετέπειτα φάσεις. Ο αλγόριθμος λοιπόν, στην αρχή κάθε φάσης, ελέγχει αν υπάρχουν τέτοιες ακμές και, αν υπάρχουν, προχωρά σε συμπτώξεις.

2.5.3 Ένα παθολογικό παράδειγμα

Ωστόσο, η αφελής εφαρμογή του RHS-Scaling με συμπτώξεις δεν δίνει αμέσως ισχυρά πολυωνυμικό αλγόριθμο. Το άρθρο παρουσιάζει στην Ενότητα 4.2 ένα παθολογικό πα-

ράδειγμα (Εικόνα 4 του άρθρου): ένα δίκτυο τεσσάρων κόμβων με προσφορές/ζητήσεις που εξαρτώνται από μια μεγάλη παράμετρο M . Σε αυτό το δίκτυο, η τυπική συμπτωτική διαδικασία δημιουργεί έναν κόμβο με εξαιρετικά μικρή προσφορά/ζήτηση αλλά με πλεόνασμα συγκρίσιμο με το Δ , γεγονός που τον αναγκάζει να αναγεννάται σε πολλές φάσεις και τελικά εκθέτει τον αλγόριθμο σε $\log M$ φάσεις.

Αυτή η παρατήρηση οδηγεί στην τροποποίηση της επιλογής πηγής και καταβόθρας που περιγράφεται στην επόμενη υποενότητα.

2.5.4 Τροποποίηση των κανόνων επαύξησης

Η τροποποίηση είναι απλή αλλά κρίσιμη: αντί να αναζητούμε κόμβους με $e(i) \geq \Delta$ και $e(i) \leq -\Delta$, αναζητούμε κόμβους με $e(i) \geq \alpha\Delta$ και $e(i) \leq -\alpha\Delta$, για μια επιλεγμένη παράμετρο $\alpha \in (1/2, 1)$. Η επιλογή αυτή εξασφαλίζει ότι κάθε αναγεννημένος κόμβος έχει πλεόνασμα πολύ κοντά σε $\alpha\Delta$, όχι οριακά κοντά στο Δ , γεγονός που σπάει το παθολογικό σενάριο.

Η υλοποίησή μας χρησιμοποιεί $\alpha = 3/4$, εκφρασμένο ως κλάσμα ώστε να αποφεύγονται οι αριθμητικές πράξεις κινητής υποδιαστολής. Συγκεκριμένα, ένας κόμβος i είναι πηγή αν $4 \cdot e(i) \geq 3 \cdot \Delta$ και είναι καταβόθρα αν $4 \cdot e(i) \leq -3 \cdot \Delta$.

2.6 Ο ισχυρά πολυωνυμικός αλγόριθμος

2.6.1 Περιγραφή αλγορίθμου

Ο ισχυρά πολυωνυμικός αλγόριθμος του Orlin (Εικόνα 5 του άρθρου) μπορεί να περιγραφεί ως εξής:

Αλγόριθμος 2: Strongly-Polynomial (Εικόνα 5 του άρθρου του Orlin)

Είσοδος: Δίκτυο $G = (V, E)$ με κόστη c και τιμές $b(\cdot)$ **Έξοδος:** Βέλτιστη ροή x στο αρχικό δίκτυο

```
1  $x \leftarrow 0$ ;  
2  $\pi \leftarrow 0$ ;  
3  $e \leftarrow b$ ;  
4  $\Delta \leftarrow \max\{e(i) : i \in V\}$ ;  
5 ενόσω υπάρχει  $i$  με  $e(i) \neq 0$  επανάλαβε  
6   αν  $x_{ij} = 0 \ \forall (i, j) \in \hat{A}$  και  $e(i) < \Delta \ \forall i \in \hat{N}$  τότε  
7      $\Delta \leftarrow \max\{e(i) : i \in \hat{N}\}$  ; // πλήρης αναγέννηση  
8   ενόσω υπάρχει ακμή  $(i, j)$  με  $x_{ij} \geq 3n\Delta$  επανάλαβε  
9     σύμπτυξη των  $i, j$  σε νέο κόμβο με ενημέρωση δυναμικών και κοστών;  
/*  $\Delta$ -φάση κλιμάκωσης */  
10  ενόσω  $\exists i \in S(\alpha\Delta), j \in T(\alpha\Delta)$  επανάλαβε  
11    υπολόγισε αποστάσεις  $d(k)$  από τον  $i$  στο  $G(x)$  με μήκη  $c^\pi$ ;  
12     $\pi(k) \leftarrow \pi(k) - d(k), \ \forall k$ ;  
13    επαύξησε  $\Delta$  μονάδες κατά μήκος του μονοπατιού  $i \rightarrow j$ ;  
14     $\Delta \leftarrow \Delta/2$ ;  
/* αναίρεση συμπτύξεων */  
15 για κάθε σύμπτυξη από την τελευταία προς την πρώτη επανάλαβε  
16   επανάφερε τον συμπτυγμένο κόμβο;  
17   ενημέρωσε δυναμικά με το αντίστροφο βήμα;  
18 υπολόγισε μέγιστη ροή στο υπο-δίκτυο  $\{(i, j) : c_{ij}^\pi = 0\}$  για την τελική εφικτή ροή;
```

2.6.2 Ορθότητα και ανάλυση πολυπλοκότητας

Η ορθότητα του αλγορίθμου ακολουθεί από τη διατήρηση της συνθήκης βελτιστότητας σε κάθε βήμα, ακριβώς όπως στον RHS-Scaling, με την επιπλέον παρατήρηση ότι η σύμπτυξη δύο κόμβων που συνδέονται με ισχυρά εφικτή ακμή διατηρεί το σύνολο βέλτιστων λύσεων του προβλήματος (Λήμματα 2 και 7 του άρθρου). Το βήμα πλήρους αναγέννησης επιτρέπεται όταν οι ροές έχουν μηδενιστεί και όλα τα πλεονάσματα βρίσκονται κάτω από το Δ : σε αυτό το σημείο ο αλγόριθμος έχει ουσιαστικά έναν αρχικό καθαρό κόμβο, και μπορεί να επανεκκινήσει με μικρότερη τιμή κλίμακας.

Θεώρημα 2.2 (Πολυπλοκότητα, Θεωρήματα 2, 3 και 4 του Orlin). *Ο συνολικός αριθμός επαυξήσεων στον ισχυρά πολυωνυμικό αλγόριθμο είναι $O(n \log n)$. Ο συνολικός αριθμός φάσεων είναι επίσης $O(n \log n)$. Κάθε επαύξηση απαιτεί έναν υπολογισμό συντομότερου μονοπατιού, οπότε ο συνολικός χρόνος είναι $O((n \log n) \cdot S(n, m))$.*

2.6.3 Αναίρεση συμπτώξεων και εξαγωγή βέλτιστης ροής

Το σχέδιο του Orlin, στην πραγματικότητα, επιλύει το δυικό: παράγει βέλτιστα δυναμικά π^* . Για να παραχθεί η βέλτιστη ροή στο αρχικό δίκτυο χρειάζονται δύο βήματα. Πρώτον, οι συμπτώξεις αναιρούνται με αντίστροφη χρονική σειρά: όταν αναιρείται μια σύμπτυξη, τα δυναμικά των δύο άκρων ορίζονται ίσα με το δυναμικό του συμπτυγμένου κόμβου, και προστίθενται οι μεταβολές δυναμικών που είχαν σημειωθεί πριν τη σύμπτυξη. Το Λήμμα 15 του άρθρου εγγυάται ότι τα παραγόμενα δυναμικά είναι βέλτιστα για το μη συμπτυγμένο δίκτυο.

Δεύτερον, δοθέντων των βέλτιστων δυναμικών π^* , κατασκευάζουμε το υπο-δίκτυο $G^\circ = (V, E^\circ)$, όπου

$$E^\circ = \{ (i, j) \in E : c_{ij}^{\pi^*} = 0 \}.$$

Η βέλτιστη ροή στο αρχικό πρόβλημα είναι μια εφικτή ροή στο G° που ικανοποιεί τις συνθήκες ισορροπίας μάζας. Ισοδύναμα, είναι ο υπολογισμός μιας μέγιστης ροής από έναν τεχνητό υπερ-πηγή s^* (συνδεδεμένο με ακμές χωρητικότητας $b(i)$ προς κάθε κόμβο με $b(i) > 0$) σε έναν τεχνητό υπερ-καταβόθρα t^* (συνδεδεμένο με ακμές χωρητικότητας $-b(i)$ από κάθε κόμβο με $b(i) < 0$).

2.7 Επέκταση σε δίκτυα με χωρητικότητες

Ο αλγόριθμος του Orlin παρουσιάζεται κυρίως για μη χωρητικά δίκτυα (δηλαδή με $u_{ij} = \infty$ για κάθε ακμή). Η επέκταση σε δίκτυα με πεπερασμένες χωρητικότητες γίνεται με κλασικό μετασχηματισμό: κάθε χωρητική ακμή (i, j) με χωρητικότητα u_{ij} αντικαθίσταται από έναν νέο κόμβο k και δύο μη χωρητικές ακμές (i, k) και (j, k) , με κατάλληλες προσαρμογές στις προσφορές/ζητήσεις (Εικόνα 6 του άρθρου).

Στην παρούσα υλοποίηση εστιάζουμε στο μη χωρητικό σενάριο, καθώς αυτό είναι που επιλύει άμεσα ο κώδικας σε C. Για χωρητικά δίκτυα, ο μετασχηματισμός μπορεί να εφαρμοστεί εξωτερικά από τον κώδικα, στο επίπεδο της γεννήτριας εισόδου.

Κεφάλαιο 3. Υλοποίηση των αλγορίθμων

3.1 Μορφή αρχείου εισόδου και αναπαράσταση δικτύου

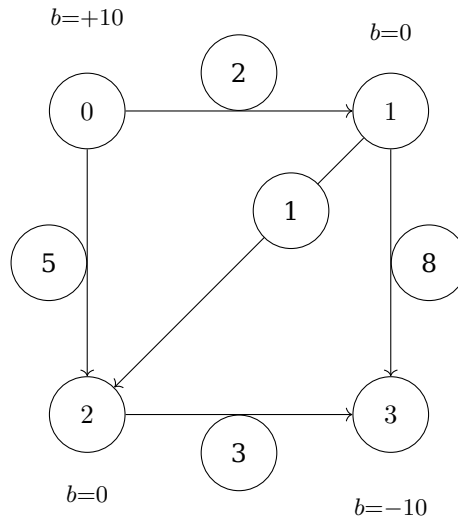
Και οι δύο υλοποιήσεις δέχονται τα δίκτυα ως αρχείο κειμένου από την τυπική είσοδο (stdin). Το αρχείο έχει την ακόλουθη δομή:

```
n m
b(0) b(1) ... b(n-1)
from_0 to_0 cost_0
from_1 to_1 cost_1
...
from_{m-1} to_{m-1} cost_{m-1}
```

Η πρώτη γραμμή περιέχει τους ακεραίους n (πλήθος κόμβων) και m (πλήθος ακμών), χωρισμένους με κενό. Η δεύτερη γραμμή περιέχει n ακεραίους, τα $b(i)$ για $i = 0, 1, \dots, n-1$. Οι επόμενες m γραμμές περιγράφουν τις ακμές: κάθε γραμμή αποτελείται από τρεις ακεραίους, τον κόμβο-ουρά, τον κόμβο-κεφαλή και το κόστος της ακμής.

Ένα παράδειγμα δικτύου με 4 κόμβους και 5 ακμές φαίνεται παρακάτω. Ο κόμβος 0 έχει προσφορά 10, ο κόμβος 3 έχει ζήτηση 10, ενώ οι ενδιάμεσοι κόμβοι είναι κόμβοι διαμεταγωγής.

```
4 5
10 0 0 -10
0 1 2
0 2 5
1 2 1
1 3 8
2 3 3
```



Σχήμα 3.1: Σχηματική αναπαράσταση του δικτύου 4 κόμβων. Οι αριθμοί στις ακμές είναι τα κόστη.

3.2 Γεννήτρια δικτύων δοκιμών

Η δημιουργία δικτύων δοκιμής γίνεται από το σενάριο `Python generator.py`. Η γεννήτρια χρησιμοποιεί τη βιβλιοθήκη `NetworkX` και κατασκευάζει ένα τυχαίο κατευθυνόμενο γράφημα με τον εξής τρόπο. Αρχικά δημιουργούνται όλα τα πιθανά διατεταγμένα ζεύγη (u, v) με $u \neq v$ και επιλέγεται ένα τυχαίο υποσύνολο μεγέθους ίσου με το μισό του συνολικού πλήθους ζευγών. Αν το παραγόμενο γράφημα δεν είναι ισχυρά συνδεδεμένο, η διαδικασία επαναλαμβάνεται.

Στη συνέχεια επιλέγονται οι πρώτοι `n_sources` κόμβοι ως πηγές και οι τελευταίοι `n_sinks` κόμβοι ως καταβόθρες. Η συνολική προσφορά `total_supply` κατανέμεται τυχαία ανάμεσα στις πηγές και η συνολική ζήτηση επίσης τυχαία ανάμεσα στις καταβόθρες. Όλοι οι ενδιάμεσοι κόμβοι ορίζονται ως κόμβοι διαμεταγωγής (με `demand = 0`).

Σε κάθε ακμή ανατίθεται ένα τυχαίο ακέραιο κόστος στο διάστημα $[1, 100]$. Το παραγόμενο γράφημα περνά από τη συνάρτηση `min_cost_flow_cost` της `NetworkX`, η οποία επιστρέφει το βέλτιστο κόστος. Αυτή η τιμή εκτυπώνεται στο τερματικό μαζί με τα υπόλοιπα στοιχεία του δικτύου και χρησιμοποιείται ως αναφορά ορθότητας όταν τρέξουμε τους δικούς μας αλγορίθμους στο ίδιο αρχείο εισόδου.

Η κλήση της γεννήτριας γίνεται από τη γραμμή εντολών ως εξής:

```
python generator.py <nodes> <sources> <sinks> <supply> <output> [--seed N]
```

Η παράμετρος `--seed` επιτρέπει τον έλεγχο της πηγής τυχαιότητας, ώστε τα πειράματα να είναι αναπαραγώγιμα.

Ο σχεδιασμός της γεννήτριας έχει δύο σημαντικές ιδιότητες. Πρώτον, το παραγόμενο δίκτυο είναι πάντα ισορροπημένο, δηλαδή $\sum_i b(i) = 0$, οπότε δεν χρειάζεται η εξωτερική παραδοχή ισορροπίας. Δεύτερον, αποφεύγει την περίπτωση δικτύων χωρίς εφικτή ροή μέσω του ελέγχου ισχυρής συνδεδεμένης.

Η συνάρτηση `export_to_file` γράφει το δίκτυο στο αρχείο εξόδου με τη μορφή που περιγράφηκε στην προηγούμενη ενότητα. Τα demands μετατρέπονται σε supplies μέσω πολλαπλασιασμού με -1 , ώστε κόμβοι με θετικές τιμές να αντιστοιχούν σε πηγές και αρνητικές σε καταβόθρες.

3.3 Υλοποίηση του RHS-Scaling (Section 3)

Η υλοποίηση του RHS-Scaling βρίσκεται στο αρχείο `main_section3.c`. Το αρχείο είναι αυτόνομο: περιέχει όλες τις δομές δεδομένων, τις βοηθητικές συναρτήσεις και τη `main` συνάρτηση, και παράγει ένα εκτελέσιμο ονόματι `mcfs_section3`.

3.3.1 Δομές δεδομένων

Η δομή `Arc` αναπαριστά μια ακμή του δικτύου:

```
typedef struct {
    int from;
    int to;
    int cost;
    int flow;
} Arc;
```

Κάθε ακμή αποθηκεύει τον κόμβο-ουρά, τον κόμβο-κεφαλή, το κόστος της και την τρέχουσα ροή της. Τα πεδία αυτά ενημερώνονται σε όλη τη διάρκεια του αλγορίθμου.

Η δομή `Graph` αναπαριστά ολόκληρο το δίκτυο μαζί με τις καταστάσεις του αλγορίθμου:

```
typedef struct {
    int n;
    int m;
    int *supplies;      /* αρχικά  $b(i)$  */
    int *excess;        /* τρέχοντα  $e(i)$  */
    int *potentials;    /* δυναμικά  $\pi(i)$  */
    int *distances;     /* αποστάσεις  $d(i)$  Dijkstra */
    int *parent_arc;    /* πατέρας στο δένδρο Dijkstra */
    int *reduced_costs; /*  $c^p$  για κάθε ακμή */
    Arc *arcs;         /* πίνακας ακμών */
    int delta;          /* τρέχον  $\Delta$  */
} Graph;
```

Η χρήση απλών πινάκων αντί για λίστες γειτνίασης είναι ηθελημένη: διευκολύνει την κατανόηση και επιτρέπει εύκολη ενημέρωση όλων των πεδίων. Το μέγεθος δεν αποτελεί πρόβλημα για τα δίκτυα δοκιμών που χρησιμοποιούμε, αφού το m είναι της τάξης n^2 .

3.3.2 Αρχικοποίηση και υπολογισμός του Δ

Η συνάρτηση `graph_create` διαβάζει το αρχείο εισόδου από τον δοσμένο δείκτη `FILE` και εκχωρεί δυναμικά τις δομές δεδομένων. Η ανάγνωση γίνεται με `fscanf` και ελέγχεται για αποτυχίες.

Η συνάρτηση `graph_init` εκτελεί την αρχικοποίηση του αλγορίθμου: θέτει $x = 0$, $\pi = 0$, $e = b$ και υπολογίζει την αρχική τιμή του Δ ως τη μικρότερη δύναμη του 2 που είναι μεγαλύτερη ή ίση με $U = \max_i |b(i)|$. Η δύναμη του 2 υπολογίζεται από τη βοηθητική `ceil_log2`:

```
static int ceil_log2(int x) {
    if (x <= 1) return 0;
    int log = 0;
    int val = 1;
    while (val < x) {
        val *= 2;
        log++;
    }
    return log;
}
```

Η τιμή του Δ στη συνέχεια αποθηκεύεται ως $1 \ll \text{ceil_log2}(U)$. Αν $U = 0$ (δηλαδή όλες οι τιμές $b(i)$ είναι μηδέν και το πρόβλημα είναι τετριμμένο), τότε $\Delta = 1$, ώστε να διακοπεί αμέσως ο κύριος βρόχος.

3.3.3 Υπολογισμός μειωμένων κοστών

Η συνάρτηση `graph_compute_reduced_costs` διατρέχει όλες τις ακμές και υπολογίζει το $c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j)$, αποθηκεύοντας τα αποτελέσματα στον πίνακα `reduced_costs`. Η συνάρτηση καλείται στην αρχή κάθε επαύξησης, πριν τον υπολογισμό του Dijkstra, ώστε ο αλγόριθμος Dijkstra να χρησιμοποιεί τα πιο πρόσφατα μειωμένα κόστη.

3.3.4 Υπολογισμός συντομότερων μονοπατιών (Dijkstra)

Η συνάρτηση `graph_compute_shortest_paths` υλοποιεί τον αλγόριθμο Dijkstra με γραμμική αναζήτηση ελαχίστου (ουσιαστικά διάταξη $O(n^2)$). Επιλέχθηκε αυτή η υλοποίηση αντί για υλοποίηση με `heap` λόγω της απλότητάς της και της άμεσης αντιστοιχίας με τη θεωρητική περιγραφή του αλγορίθμου.

Δύο στοιχεία αξίζει να σημειωθούν. Πρώτον, ο Dijkstra εκτελείται στο υπολειπόμενο δίκτυο, όχι στο αρχικό δίκτυο. Συνεπώς, για κάθε ακμή (i, j) του αρχικού δικτύου, εξε-

τάζονται τόσο η προσθετική της εκδοχή (i, j) με κόστος c_{ij}^π , όσο και η αφαιρετική της (j, i) με κόστος $-c_{ij}^\pi$, αλλά η αφαιρετική υπάρχει μόνο αν η τρέχουσα ροή είναι θετική. Δεύτερον, ο πίνακας `parent_arc` αποθηκεύει τον δείκτη της ακμής που χρησιμοποιήθηκε στο συντομότερο μονοπάτι. Για τις προσθετικές ακμές ο δείκτης είναι απλώς μη αρνητικός· για τις αφαιρετικές ακμές, κωδικοποιούμε τον δείκτη i ως $-(i+1)$, ώστε το πρόσημο να διακρίνει την κατεύθυνση.

Για κάθε κόμβο που εξάγεται ως τρέχων ελάχιστος, η συνάρτηση κάνει χαλάρωση όλων των ακμών του αρχικού πίνακα: ελέγχει αν η ουρά της τρέχουσας ακμής είναι ο εξαγόμενος κόμβος (προσθετική ακμή) ή η κεφαλή της (αφαιρετική ακμή), και ενημερώνει την απόσταση του γείτονα αν προκύπτει μικρότερη.

3.3.5 Ενημέρωση δυναμικών κόμβων

Η συνάρτηση `graph_update_potentials` ενημερώνει τα δυναμικά σύμφωνα με το Λήμμα ενημέρωσης δυναμικών: $\pi(i) := \pi(i) - d(i)$ για κάθε κόμβο i . Κόμβοι που δεν προσεγγίζονται από τον Dijkstra (`distances[i] == INF`) παραμένουν αμετάβλητοι, καθώς η μη πεπερασμένη τιμή δεν έχει θεωρητική σημασία για τη συντήρηση της συνθήκης βελτιστότητας στο συνεκτικό τμήμα του δικτύου.

3.3.6 Επαύξηση ροής

Η συνάρτηση `graph_augment_flow` διατρέχει το συντομότερο μονοπάτι από την καταβόθρα προς την πηγή, ακολουθώντας τον πίνακα `parent_arc`. Για κάθε ακμή στο μονοπάτι, αυξάνει τη ροή κατά Δ αν πρόκειται για προσθετική ακμή και μειώνει τη ροή κατά Δ αν πρόκειται για αφαιρετική. Στο τέλος, μειώνει το $e(\text{source})$ κατά Δ και αυξάνει το $e(\text{sink})$ κατά Δ .

3.3.7 Κύριος βρόχος αλγορίθμου

Η συνάρτηση `graph_run_main_loop` υλοποιεί τον κύριο δομικό βρόχο του RHS-Scaling. Ο εξωτερικός βρόχος εκτελείται όσο $\Delta \geq 1$. Μέσα σε κάθε φάση, ο εσωτερικός βρόχος αναζητά διαδοχικά ζεύγη (πηγή, καταβόθρα) και εκτελεί επαύξηση, μέχρι κάποιο από τα $S(\Delta)$, $T(\Delta)$ να αδειάσει. Αν, σε οποιαδήποτε στιγμή, η καταβόθρα δεν είναι προσβάσιμη από την πηγή στο υπολειπόμενο δίκτυο, ο αλγόριθμος επιστρέφει αποτυχία (που σημαίνει ότι το πρόβλημα δεν έχει εφικτή λύση).

Οι συναρτήσεις `graph_find_source` και `graph_find_sink` διατρέχουν τον πίνακα πλεονασμάτων και επιστρέφουν τον πρώτο κόμβο με $e(i) \geq \Delta$ ή $e(i) \leq -\Delta$ αντίστοιχα, ή -1 αν δεν υπάρχει.

Τέλος, η `graph_get_total_cost` υπολογίζει το συνολικό κόστος της τρέχουσας ροής ως $\sum c_{ij} \cdot x_{ij}$. Το αποτέλεσμα τυπώνεται από τη `main`.

3.4 Υλοποίηση του ισχυρά πολυωνυμικού αλγορίθμου (Section 4)

Η υλοποίηση του ισχυρά πολυωνυμικού αλγορίθμου βρίσκεται στο αρχείο `main_section4.c`. Η βασική δομή ακολουθεί την αντίστοιχη του Section 3, αλλά εμπλουτίζεται με μηχανισμούς σύμπτυξης, αναίρεσης συμπτύξεων, επιλογής πηγής/καταβόθρας με παράμετρο α , πλήρους αναγέννησης και τελικού υπολογισμού βέλτιστης ροής μέσω μέγιστης ροής.

3.4.1 Επέκταση δομών δεδομένων

Η δομή `Node` ενσωματώνει όλες τις πληροφορίες κάθε κόμβου:

```
typedef struct {
    int supply;           /*  $b(i)$  τρέχουσα τιμή */
    int original_supply;  /*  $b(i)$  αρχική τιμή */
    int excess;          /*  $e(i)$  πλεόνασμα */
    int potential;        /*  $p(i)$  δυναμικό */
    int distance;         /*  $d(i)$  απόσταση Dijkstra */
    int parent_arc;       /* γονέας στο δένδρο */
    bool active;          /* ενεργός (όχι συμπτυγμένος) */
} Node;
```

Η διάκριση μεταξύ `supply` και `original_supply` είναι κρίσιμη: όταν γίνεται σύμπτυξη δύο κόμβων, το `supply` του «επιζώντος» κόμβου ενημερώνεται ώστε να περιλαμβάνει τη συνολική προσφορά/ζήτηση των δύο αρχικών. Το `original_supply` διατηρεί την αρχική τιμή και χρησιμοποιείται στον τελικό υπολογισμό μέγιστης ροής.

Η δομή `Arc` επεκτείνεται με πληροφορίες για τα αρχικά άκρα και το αρχικό κόστος:

```
typedef struct {
    int from, to, cost, flow;
    int original_from, original_to, original_cost;
} Arc;
```

Μετά από σύμπτυξη, τα πεδία `from` και `to` ενδέχεται να αντικατασταθούν (ώστε όλες οι ακμές να δείχνουν στον επιζώντα κόμβο αντί στον απορροφημένο). Τα πεδία `original_from` και `original_to` διατηρούν την αρχική τοπολογία, ώστε να είναι δυνατή η αναίρεση στο τέλος.

Η δομή `ContractionRecord` καταγράφει κάθε σύμπτυξη και είναι απαραίτητη για τη σωστή αναίρεση:

```
typedef struct {
    int node_k, node_l;
    int *potentials_snapshot;
    int n;
} ContractionRecord;
```

Εδώ ο `node_k` είναι ο επιζών κόμβος, ο `node_l` είναι ο απορροφημένος, και `potentials_snapshot` είναι ένα στιγμιότυπο των δυναμικών όλων των κόμβων τη στιγμή της σύμπτυξης. Αυτό χρησιμοποιείται αργότερα, κατά την αναίρεση, για να αποκατασταθούν σωστά τα δυναμικά. Τέλος, η κεντρική δομή `Graph` επεκτείνεται με ένα δυναμικό ιστορικό συμπτύξεων:

```
typedef struct {
    int n, m;
    Node *nodes;
    Arc *arcs;
    int delta;
    int n_active;
    ContractionRecord *history;
    int history_count;
    int history_capacity;
} Graph;
```

Το `n_active` παρακολουθεί τον τρέχοντα αριθμό ενεργών κόμβων, μειούμενο κατά μία μονάδα σε κάθε σύμπτυξη. Το `history` είναι δυναμικά μεταβλητού μεγέθους και διπλασιάζεται όταν γεμίζει, τυπική τεχνική αμορτισμένου γραμμικού κόστους.

3.4.2 Διαδικασία σύμπτυξης κόμβων

Η συνάρτηση `graph_contract` υλοποιεί τη σύμπτυξη δύο κόμβων k και l , με τον k να είναι ο επιζών και τον l να απορροφάται. Τα βήματά της είναι:

1. Καταγραφή της σύμπτυξης στο ιστορικό: δημιουργία νέου `ContractionRecord` με αποθήκευση των δεικτών k και l και στιγμιότυπου όλων των τρεχόντων δυναμικών.
2. Μετατροπή των κοστών σε μειωμένα κόστη: για κάθε ακμή (i, j) , θέτουμε $c := c - \pi(i) + \pi(j)$. Αυτό αποτυπώνει στο επίπεδο ακμών τις πληροφορίες που κωδικοποιούνται στα τρέχοντα δυναμικά, ώστε στη συνέχεια τα δυναμικά να μπορούν να μηδενιστούν χωρίς απώλεια πληροφορίας.
3. Μηδενισμός των δυναμικών όλων των κόμβων.
4. Συγχώνευση των `supplies` και των `excess`: `supply[k] += supply[l]`, `excess[k] += excess[l]`.
5. Αντικατάσταση του l με τον k σε όλες τις ακμές: κάθε ακμή που είχε `from == l`

πλέον έχει $\text{from} = k$, και ομοίως για $\text{to} == l$.

6. Απενεργοποίηση του l : $\text{nodes}[l].\text{active} = \text{false}$, και μείωση του n_{active} .

Σημειώνουμε ότι μετά από σύμπτυξη δημιουργούνται συνήθως πολλαπλές ακμές (δύο ακμές με τους ίδιους ακραίους κόμβους), καθώς και βρόχοι (ακμές $k \rightarrow k$ που προέρχονται από την πρώην $k \rightarrow l$). Αυτές οι περιπτώσεις αντιμετωπίζονται εξαρτημένα από τη λογική κάθε επόμενης συνάρτησης.

3.4.3 Αναγνώριση και σύμπτυξη ισχυρά εφικτών ακμών

Η συνάρτηση `graph_contract_strongly_feasible` αναζητά επαναληπτικά ακμές με ροή $\geq 3n\Delta$ και συμπύκνωση τα άκρα τους. Ο έλεγχος παραλείπει ακμές που έχουν γίνει βρόχοι ($\text{from} == \text{to}$) και ακμές που συνδέονται με ήδη απενεργοποιημένους κόμβους. Η επανάληψη σταματά όταν καμία νέα ισχυρά εφικτή ακμή δεν βρεθεί σε μια πλήρη διέλευση.

Το κατώφλι $3n\Delta$ υλοποιείται ως $(\text{long long})3 * g->n * g->\text{delta}$, με ρητή μετατροπή σε 64-bit ακέραιο ώστε να αποφευχθεί υπερχείλιση για μεγαλύτερα δίκτυα.

3.4.4 Επιλογή πηγής και καταβόθρας με κατώφλι α

Η επιλογή πηγής και καταβόθρας υλοποιείται με τις συναρτήσεις `graph_find_source` και `graph_find_sink`, αντίστοιχα. Αντί για άμεση σύγκριση με το Δ , χρησιμοποιείται η συνθήκη με την παράμετρο $\alpha = 3/4$.

```
#define ALPHA_NUM 3
#define ALPHA_DEN 4

static int graph_find_source(Graph *g) {
    for (int i = 0; i < g->n; i++) {
        Node *node = &g->nodes[i];
        if (node->active &&
            (long long)ALPHA_DEN * node->excess
            >= (long long)ALPHA_NUM * g->delta)
            return i;
    }
    return -1;
}
```

Η συνθήκη $e(i) \geq (3/4)\Delta$ εκφράζεται ισοδύναμα ως $4 \cdot e(i) \geq 3 \cdot \Delta$, αποφεύγοντας διαιρέσεις και πράξεις κινητής υποδιαστολής.

Αντίστοιχα, η `graph_find_sink` ελέγχει αν $4 \cdot e(i) \leq -3 \cdot \Delta$. Και οι δύο συναρτήσεις εξετάζουν μόνο ενεργούς (μη συμπυκνώνους) κόμβους.

3.4.5 Πλήρης αναγέννηση

Το βήμα πλήρους αναγέννησης ενεργοποιείται όταν πληρούνται ταυτόχρονα δύο συνθήκες: (α) όλες οι ροές του δικτύου είναι μηδενικές (με εξαίρεση τις ακμές-βρόχους που προκύπτουν από συμπτώξεις) και (β) όλα τα πλεονάσματα των ενεργών κόμβων είναι αυστηρά κάτω από το Δ .

Η υλοποίηση διακρίνει δύο βοηθητικές συναρτήσεις:

```
static bool all_flows_zero(Graph *g) {
    for (int i = 0; i < g->m; i++) {
        if (g->arcs[i].from != g->arcs[i].to &&
            g->arcs[i].flow != 0)
            return false;
    }
    return true;
}

static bool all_excesses_below_delta(Graph *g) {
    for (int i = 0; i < g->n; i++) {
        if (g->nodes[i].active &&
            g->nodes[i].excess >= g->delta)
            return false;
    }
    return true;
}
```

Όταν ενεργοποιηθεί η αναγέννηση, υπολογίζεται η μέγιστη θετική τρέχουσα τιμή πλεονάσματος και το Δ αναπροσαρμόζεται στη μικρότερη δύναμη του 2 που είναι μεγαλύτερη ή ίση με αυτήν την τιμή. Αυτό επιτρέπει μεγαλύτερα βήματα αναπήδησης στις τιμές κλίμακας και είναι κρίσιμο για την ισχυρά πολωνυμική εγγύηση του αλγορίθμου.

3.4.6 Αναίρεση συμπτώξεων

Η συνάρτηση `graph_uncontract_all` εκτελείται μετά τη σύγκλιση του κύριου αλγορίθμου. Διατρέχει το ιστορικό συμπτώξεων με αντίστροφη χρονική σειρά και αποκαθιστά τους συμπτυγμένους κόμβους. Για κάθε εγγραφή (k, l) :

1. Ενεργοποιεί εκ νέου τον κόμβο l και αυξάνει τον `n_active`.
2. Αντιγράφει τα τρέχοντα δυναμικά του k στον l (ώστε να αποτυπώνεται η ιδιότητα $\pi^*(k) = \pi^*(l)$, όπως εγγυάται το Λήμμα 7 του άρθρου).
3. Προσθέτει το στιγμιότυπο δυναμικών που είχε καταγραφεί τη στιγμή της σύμπτυξης σε όλους τους κόμβους. Αυτό αντιστρέφει τον μηδενισμό δυναμικών που είχε γίνει κατά τη σύμπτυξη.

Μετά την ολοκλήρωση της αναίρεσης, επαναφέρονται τα αρχικά άκρα των ακμών από τα `original_from` και `original_to`. Το τελικό αποτέλεσμα είναι ότι ο γράφος επανέρχεται στην αρχική τοπολογία του, ενώ τα δυναμικά περιέχουν τις βέλτιστες τιμές.

3.4.7 Υπολογισμός βέλτιστης ροής μέσω μέγιστης ροής

Το τελικό βήμα, όπως προβλέπεται στην Ενότητα 4.5 του άρθρου, είναι ο υπολογισμός βέλτιστης ροής στο υπο-δίκτυο $G^\circ = \{(i, j) : c_{ij}^{\pi^*} = 0\}$ που ικανοποιεί τις συνθήκες ισορροπίας μάζας.

Η υλοποίηση εφαρμόζει την κλασική μέθοδο Ford-Fulkerson. Εισάγονται δύο εικονικοί κόμβοι, ο υπερ-πηγή (με δείκτη n) και ο υπερ-καταβόθρα (με δείκτη $n+1$). Για κάθε κόμβο i με θετικό `original_supply` ορίζεται υπολειπόμενη χωρητικότητα `residual_src[i] = supply[i]` (αντιστοιχεί σε μια ακμή από τον υπερ-πηγή). Για κάθε κόμβο με αρνητικό `original_supply` ορίζεται `residual_sink[i] = -supply[i]` (αντιστοιχεί σε μια ακμή προς τον υπερ-καταβόθρα).

Στη συνέχεια η `maxflow_bfs` εκτελεί BFS στον χώρο της αναζήτησης αυξητικού μονοπατιού:

- Από τον υπερ-πηγή μπορούμε να πηδήξουμε σε οποιονδήποτε κόμβο με `residual_src > 0`.
- Από έναν κοινό κόμβο ακολουθούμε είτε ακμή μηδενικού μειωμένου κόστους προς τα εμπρός (`EDGE_FORWARD`), είτε ακμή με θετική τρέχουσα ροή προς τα πίσω (`EDGE_REVERSE`).
- Φτάνουμε στον υπερ-καταβόθρα μόλις συναντηθεί κόμβος με `residual_sink > 0`.

Κάθε φορά που βρίσκεται μονοπάτι, υπολογίζεται το `bottleneck` — η ελάχιστη χωρητικότητα του — και η ροή αυξάνεται κατά αυτήν την τιμή. Για τις προσθετικές ακμές του G° η χωρητικότητα θεωρείται άπειρη (καθώς το αρχικό πρόβλημα είναι μη χωρητικό), για τις αφαιρετικές είναι ίση με την τρέχουσα ροή, ενώ για τις εικονικές ακμές ίση με τα `residual_src` ή `residual_sink` αντίστοιχα.

Η διαδικασία επαναλαμβάνεται ως ότου δεν υπάρχει πλέον αυξητικό μονοπάτι. Τότε οι ροές του πίνακα `arcs` αντιπροσωπεύουν μια βέλτιστη εφικτή ροή στο αρχικό δίκτυο, η οποία χρησιμοποιείται για τον υπολογισμό του συνολικού κόστους.

Η `graph_get_total_cost` παραμένει απλή: υπολογίζει $\sum x_{ij} \cdot \text{original_cost}_{ij}$, χρησιμοποιώντας τα αρχικά κόστη (όχι τα μειωμένα), ώστε το αποτέλεσμα να αναφέρεται στο αρχικό πρόβλημα.

3.5 Παράδειγμα εκτέλεσης βήμα προς βήμα

Για να γίνει πιο απτή η λειτουργία των δύο υλοποιήσεων, περιγράφουμε συνοπτικά την εκτέλεσή τους σε ένα μικρό δίκτυο δοκιμής. Θεωρούμε το δίκτυο τεσσάρων κόμβων που χρησιμοποιήθηκε ως παράδειγμα εισόδου προηγουμένως: κόμβος 0 με $b(0) = 10$, κόμβοι

1 και 2 με $b = 0$, κόμβος 3 με $b(3) = -10$, και ακμές $(0, 1)$ cost 2, $(0, 2)$ cost 5, $(1, 2)$ cost 1, $(1, 3)$ cost 8, $(2, 3)$ cost 3.

Για τον RHS-Scaling, η αρχική τιμή του Δ είναι 16 (μικρότερη δύναμη του $2 \geq U = 10$). Στην πρώτη φάση δεν υπάρχει κόμβος με $e(i) \geq 16$ ούτε $e(i) \leq -16$, οπότε η φάση λήγει αμέσως. Στη φάση $\Delta = 8$, ο κόμβος 0 έχει $e(0) = 10 \geq 8$ και ο κόμβος 3 έχει $e(3) = -10 \leq -8$. Υπολογίζεται συντομότερο μονοπάτι από τον 0 στον 3, το οποίο (στο αρχικό στάδιο $\pi = 0$) είναι το $(0 \rightarrow 1 \rightarrow 2 \rightarrow 3)$ με κόστος 6. Γίνεται επαύξηση 8 μονάδων, οπότε $e(0) = 2$ και $e(3) = -2$. Η φάση $\Delta = 4$ παραλείπεται (κανένα $|e(i)| \geq 4$). Στη φάση $\Delta = 2$ γίνεται μία ακόμη επαύξηση 2 μονάδων κατά μήκος του ίδιου μονοπατιού, μηδενίζοντας τα πλεονάσματα. Το τελικό κόστος είναι $10 \times 2 + 10 \times 1 + 10 \times 3 = 60$.

	$e(0)$	$e(1)$	$e(2)$	$e(3)$	Ροές
Αρχική κατάσταση	+10	0	0	-10	όλες 0
Μετά $\Delta = 16$	+10	0	0	-10	(καμία αλλαγή)
Μετά $\Delta = 8$	+2	0	0	-2	$x_{01}=8, x_{12}=8, x_{23}=8$
Μετά $\Delta = 4$	+2	0	0	-2	(καμία αλλαγή)
Μετά $\Delta = 2$	0	0	0	0	$x_{01}=10, x_{12}=10, x_{23}=10$

Σχήμα 3.2: Εξέλιξη πλεονασμάτων και ροών ανά φάση κλιμάκωσης.

Για τον ισχυρά πολωνυμικό αλγόριθμο, η συμπεριφορά στο συγκεκριμένο μικρό δίκτυο είναι σχεδόν ίδια με του RHS-Scaling, καθώς δεν υπάρχει χρόνος για να προκύψουν ισχυρά εφικτές ακμές πριν μηδενιστούν τα πλεονάσματα. Σε μεγαλύτερα δίκτυα με μεγαλύτερες προσφορές/ζητήσεις οι συμπτώξεις αρχίζουν να παίζουν ρόλο, όπως θα φανεί στα πειράματα του Κεφαλαίου 4.

Κεφάλαιο 4. Πειραματική Αξιολόγηση

4.1 Μεθοδολογία πειραμάτων

Στο κεφάλαιο αυτό παρουσιάζεται η πειραματική αξιολόγηση των δύο υλοποιήσεων (mcf_section3 και mcf_section4) σε τυχαία παραγόμενα δίκτυα. Στόχος είναι, αφενός, η πιστοποίηση της ορθότητας και, αφετέρου, η σύγκριση της χρονικής συμπεριφοράς τους.

Η μεθοδολογία περιλαμβάνει τα εξής βήματα για κάθε πείραμα:

1. Κλήση της γεννήτριας `generator.py` με συγκεκριμένες παραμέτρους (`n_nodes`, `n_sources`, `n_sinks`, `total_supply`, `output_file`, `seed`).
2. Η γεννήτρια γράφει το δίκτυο στο `output_file` και εκτυπώνει το βέλτιστο κόστος όπως το υπολογίζει η `NetworkX` (`min_cost_flow_cost`).
3. Εκτέλεση `./mcf_section3 < output_file` και μέτρηση χρόνου.
4. Εκτέλεση `./mcf_section4 < output_file` και μέτρηση χρόνου.
5. Σύγκριση των δύο τυπωμένων τιμών κόστους με την αναφορά της `NetworkX`.

Όλες οι μετρήσεις χρόνου γίνονται σε ένα και το αυτό σύστημα, ώστε η σύγκριση μεταξύ των υλοποιήσεων να είναι δίκαιη. Για κάθε μέγεθος δικτύου χρησιμοποιούνται πολλαπλά `seeds`, και αναφέρονται μέσοι όροι.

Πίνακας 4.1: Τεχνικά χαρακτηριστικά συστήματος πειραματικής αξιολόγησης.

Στοιχείο	Περιγραφή
Επεξεργαστής	AMD Ryzen 7 8745HS w/ Radeon 780M Graphics
Μνήμη RAM	27.2 GB
Λειτουργικό σύστημα	Linux-6.19.8-x86_64-with-glibc2.42
Μεταγλωττιστής	gcc (GCC) 15.2.0
Python	3.13.12
NetworkX	3.6.1

4.2 Έλεγχος ορθότητας με NetworkX

Ο έλεγχος ορθότητας γίνεται συγκρίνοντας τα βέλτιστα κόστη που αναφέρουν οι δύο υλοποιήσεις με την αναφορά της `NetworkX`. Επειδή και οι τρεις μέθοδοι (`NetworkX`,

RHS-Scaling, strongly polynomial) επιλύουν το ίδιο πρόβλημα και το βέλτιστο κόστος είναι μοναδικό, οι τρεις τιμές πρέπει να ταυτίζονται.

Για κάθε πείραμα επαληθεύεται η ισότητα

$$\text{cost}_{\text{NetworkX}} = \text{cost}_{\text{Section 3}} = \text{cost}_{\text{Section 4}}.$$

Σε περίπτωση μη ταύτισης σε κάποια εκτέλεση, το πείραμα σημειώνεται ως αποτυχία. Τα αποτελέσματα συνοψίζονται στον Πίνακα 4.2.

Πίνακας 4.2: Έλεγχος ορθότητας: ταύτιση βέλτιστου κόστους με NetworkX.

n	RHS-Scaling	Ισχυρά πολυωνυμικός
5	5/5	5/5
10	5/5	5/5
20	5/5	5/5
50	5/5	5/5
100	5/5	5/5

Σημείωση. Η σύγκριση των ροών ανά ακμή δεν είναι σκόπιμη σε γενική περίπτωση, καθώς το βέλτιστο πρόβλημα μπορεί να έχει πολλαπλές εναλλακτικές βέλτιστες ροές. Η μόνη αξιόπιστη έλεγχος ορθότητας είναι η ταύτιση του συνολικού κόστους.

4.3 Πειραματικά αποτελέσματα

Στις υποενότητες που ακολουθούν εξετάζεται η συμπεριφορά των δύο αλγορίθμων ως προς τρεις παραμέτρους: το μέγεθος του δικτύου (n), τον αριθμό πηγών και καταβόθρων, και το εύρος τιμών προσφοράς/ζήτησης (U). Κάθε πείραμα εκτελείται τόσο σε πυκνά δίκτυα ($m \approx n(n-1)/2$) όσο και σε αραιά ($m \approx 3n$), ώστε να αποτυπωθεί η επίδραση της πυκνότητας.

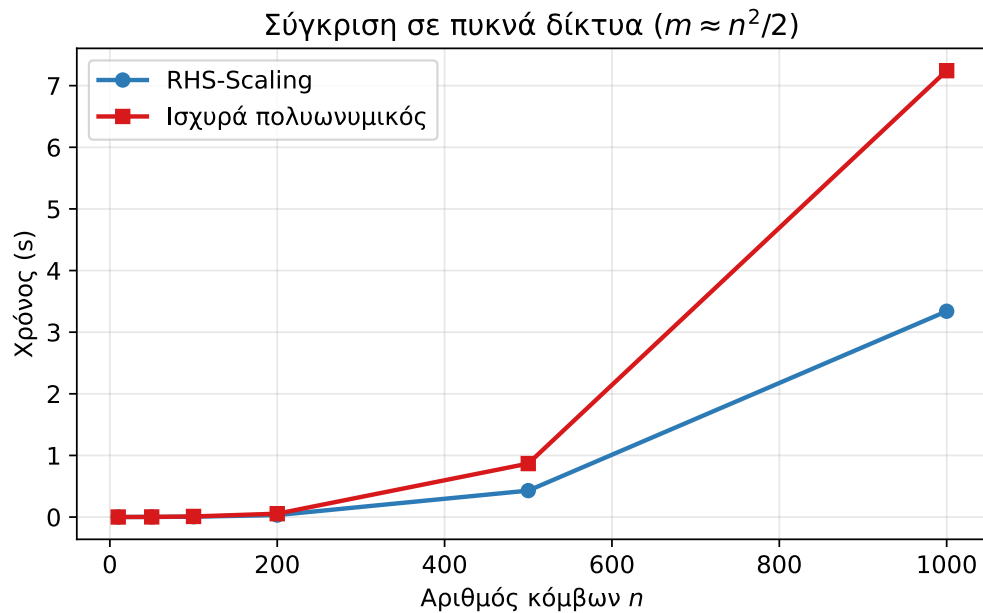
4.3.1 Επίδραση του μεγέθους δικτύου

Στο πρώτο πείραμα μεταβάλλεται ο αριθμός κόμβων n από 10 έως 1000, με μία πηγή, μία καταβόθρα και συνολική προσφορά 100. Η γεννήτρια παράγει πυκνά δίκτυα με $m = n(n-1)/2$ ακμές (Πίνακας 4.3, Σχήμα 4.1) και αραιά δίκτυα με $m \approx 3n$ ακμές (Πίνακας 4.4, Σχήμα 4.2).

Και στα δύο σενάρια ο RHS-Scaling είναι ταχύτερος για μικρές τιμές U . Στα πυκνά δίκτυα ο λόγος χρόνων αυξάνεται σταδιακά με το n , φτάνοντας ~ 2.1 για $n = 1000$. Στα αραιά ο λόγος κυμαίνεται στο ~ 1.8 για $n = 1000$. Η διαφορά στους απόλυτους χρόνους είναι μεγάλη: για $n = 1000$ τα πυκνά δίκτυα χρειάζονται ~ 3.3 s (RHS-Scaling) ενώ τα αραιά μόλις ~ 25 ms, αφού η υλοποίηση Dijkstra σαρώνει m ακμές ανά γύρο και η πολυπλοκότητα ανά κλήση είναι $O(nm)$.

Πίνακας 4.3: Χρόνοι εκτέλεσης σε πυκνά δίκτυα (μέσος όρος 5 εκτελέσεων).

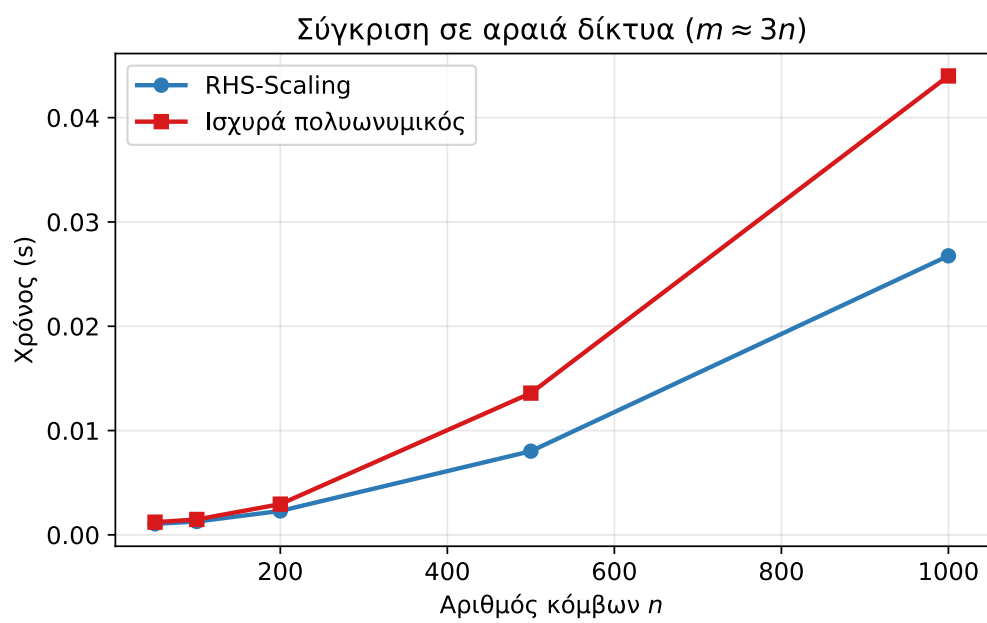
n	m	RHS-Scaling	Ισχυρά πολυωνυμικός
10	45	1.03 ms	1.05 ms
50	1225	1.99 ms	2.46 ms
100	4950	5.89 ms	10.11 ms
200	19900	32.39 ms	54.87 ms
500	124750	429.18 ms	868.19 ms
1000	499500	3.340 s	7.243 s



Σχήμα 4.1: Χρόνοι εκτέλεσης σε πυκνά δίκτυα ως συνάρτηση του n .

Πίνακας 4.4: Χρόνοι εκτέλεσης σε αραιά δίκτυα, $m \approx 3n$ (μέσος όρος 5 εκτελέσεων).

n	m	RHS-Scaling	Ισχυρά πολυωνυμικός
50	150	1.06 ms	1.22 ms
100	300	1.28 ms	1.48 ms
200	600	2.29 ms	2.94 ms
500	1500	8.02 ms	13.59 ms
1000	3000	26.75 ms	44.00 ms



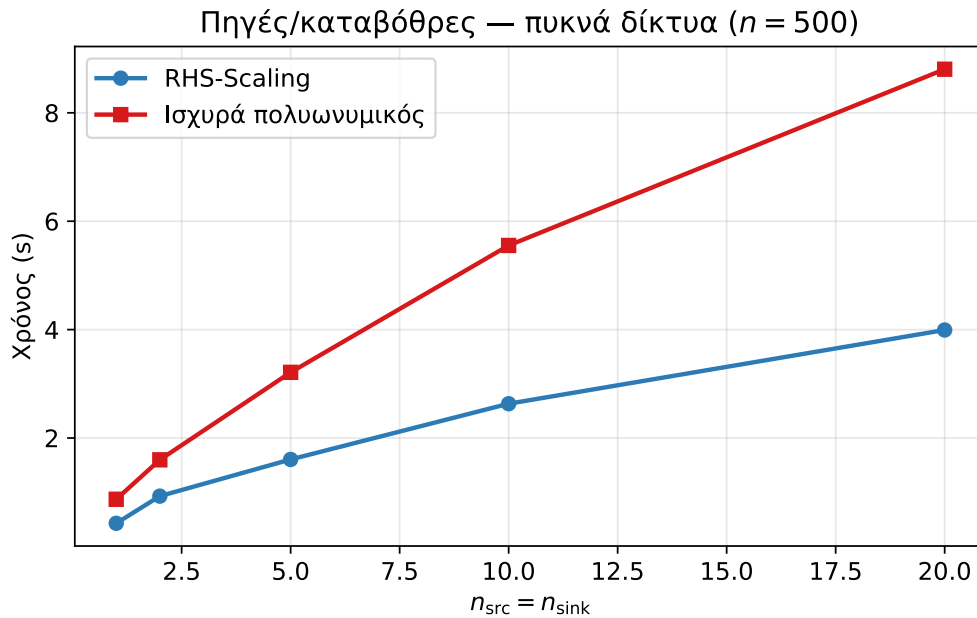
Σχήμα 4.2: Χρόνοι εκτέλεσης σε αραιά δίκτυα ($m \approx 3n$).

4.3.2 Επίδραση του αριθμού πηγών και καταβοθρών

Στο δεύτερο πείραμα διατηρείται σταθερό μέγεθος $n = 500$ και μεταβάλλεται ο αριθμός πηγών και καταβοθρών ($n_{\text{src}} = n_{\text{sink}}$) από 1 έως 20, με συνολική προσφορά 100.

Πίνακας 4.5: Πηγές/καταβόθρες σε πυκνά δίκτυα ($n = 500$, μέσος όρος 5 εκτελέσεων).

$n_{\text{src}} = n_{\text{sink}}$	RHS-Scaling	Ισχυρά πολυωνυμικός
1	426.37 ms	870.01 ms
2	927.92 ms	1.598 s
5	1.605 s	3.212 s
10	2.633 s	5.553 s
20	3.993 s	8.804 s

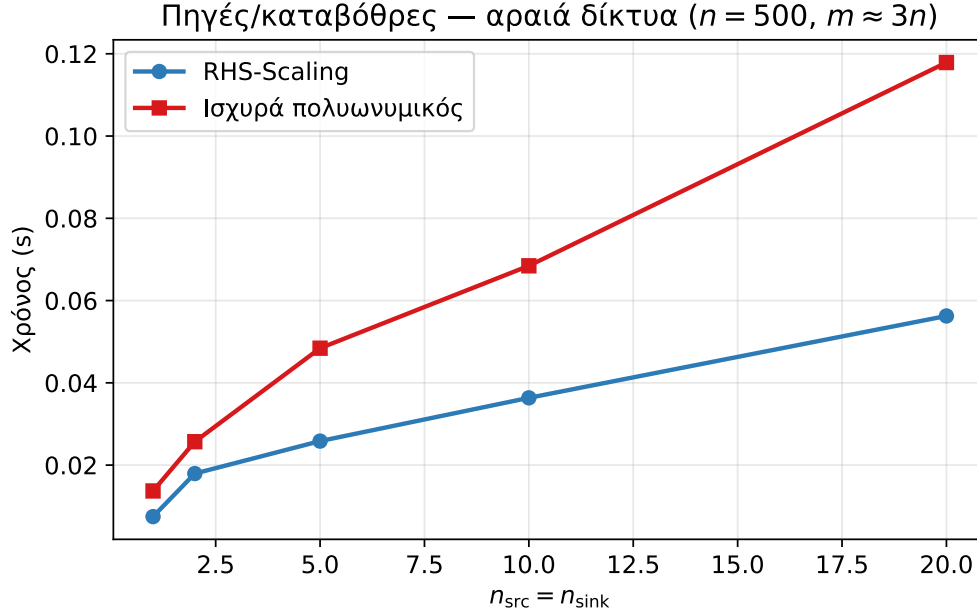


Σχήμα 4.3: Επίδραση πηγών/καταβοθρών σε πυκνά δίκτυα ($n = 500$).

Περισσότερες πηγές και καταβόθρες σημαίνει περισσότερους κόμβους που πρέπει να ισορροπηθούν σε κάθε Δ -φάση, άρα περισσότερες επαυξήσεις. Αυτό φαίνεται καθαρά στα δεδομένα: με $k = 20$ ο χρόνος είναι περίπου δεκαπλάσιος σε σχέση με $k = 1$ και στα δύο σενάρια πυκνότητας. Ο λόγος χρόνων μεταξύ των δύο αλγορίθμων παραμένει σταθερός γύρω στο $\sim 2\times$, ανεξάρτητα από τον αριθμό πηγών/καταβοθρών.

Πίνακας 4.6: Πηγές/καταβόθρες σε αραιά δίκτυα ($n = 500$, $m \approx 3n$, μέσος όρος 5 εκτελέσεων).

$n_{\text{src}} = n_{\text{sink}}$	RHS-Scaling	Ισχυρά πολωνυμικός
1	7.47 ms	13.71 ms
2	17.94 ms	25.69 ms
5	25.84 ms	48.42 ms
10	36.35 ms	68.45 ms
20	56.26 ms	117.87 ms

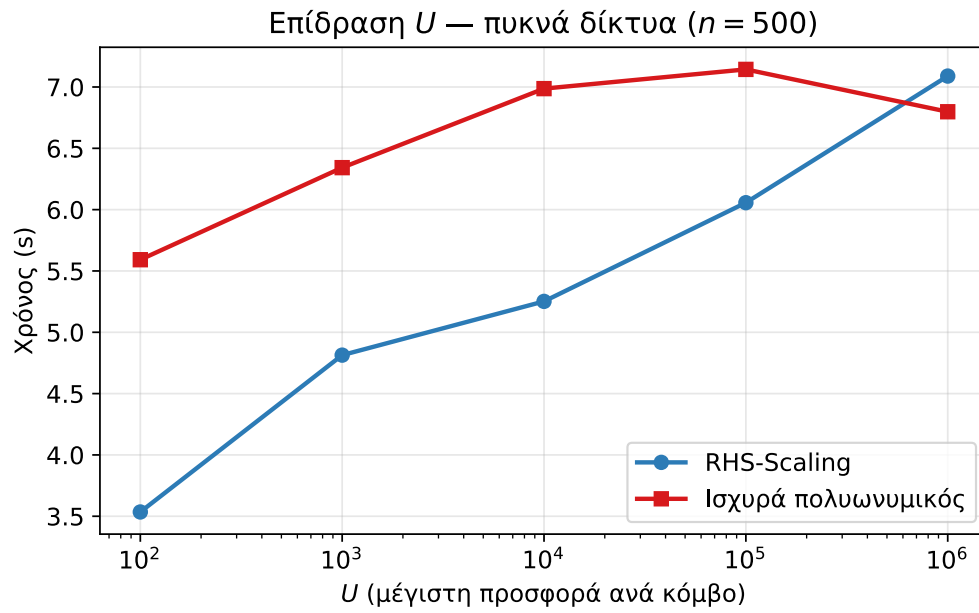


Σχήμα 4.4: Επίδραση πηγών/καταβοθρών σε αραιά δίκτυα ($n = 500$, $m \approx 3n$).

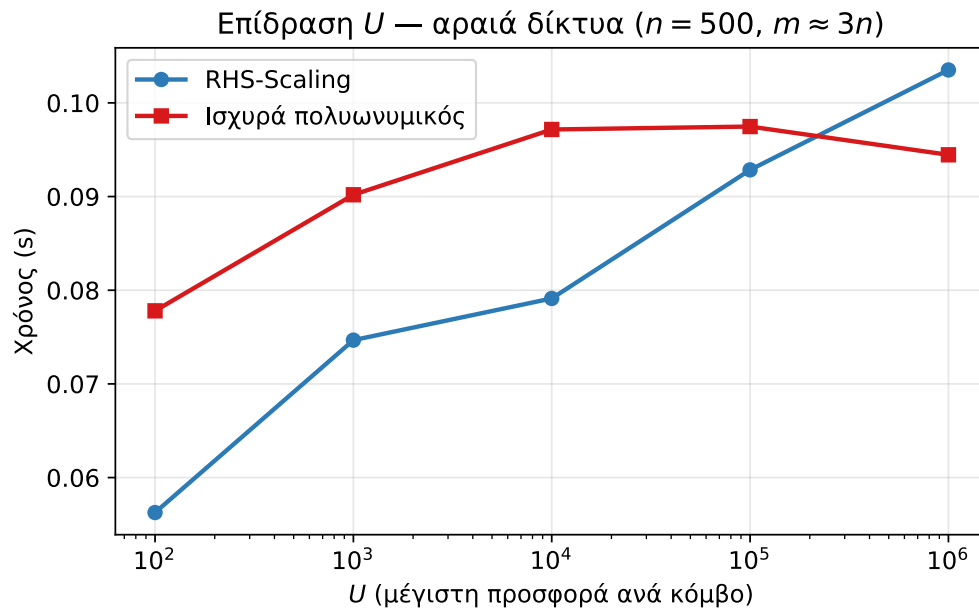
4.3.3 Επίδραση του εύρους τιμών προσφοράς/ζήτησης

Στο τρίτο πείραμα διατηρούνται σταθεροί $n = 500$ κόμβοι και 5 πηγές/καταβόθρες, ενώ μεταβάλλεται η συνολική προσφορά $U \in \{100, 10^3, 10^4, 10^5, 10^6\}$, χωρίς αλλαγή στην τοπολογία. Τα αποτελέσματα παρουσιάζονται στο Σχήμα 4.5 (πυκνά) και στο Σχήμα 4.6 (αραιά).

Ο RHS-Scaling δείχνει μονοτονική αύξηση με το U , σύμφωνα με τον παράγοντα $\log U$ στην πολυπλοκότητά του. Ο ισχυρά πολωνυμικός αλγόριθμος αυξάνεται αρχικά αλλά για $U = 10^6$ ο χρόνος του σταθεροποιείται ή μειώνεται, χάρη στη σύμπτυξη ακμών που μειώνει το ενεργό μέγεθος του γραφήματος. Στο $U = 10^6$ ο ισχυρά πολωνυμικός γίνεται ταχύτερος τόσο στα πυκνά όσο και στα αραιά δίκτυα. Αυτό το αποτέλεσμα συμφωνεί με τη θεωρητική ανάλυση: για αρκετά μεγάλο U ο παράγοντας $\log n$ του ισχυρά πολωνυμικού υπερτερεί του $\log U$ του RHS-Scaling.



Σχήμα 4.5: Χρόνοι εκτέλεσης ως συνάρτηση του U σε πυκνά δίκτυα ($n = 500$).



Σχήμα 4.6: Χρόνοι εκτέλεσης ως συνάρτηση του U σε αραιά δίκτυα ($n = 500, m \approx 3n$).

4.4 Σύνοψη πειραματικών αποτελεσμάτων

Από τα πειράματα αυτού του κεφαλαίου προκύπτουν τρία συμπεράσματα. Πρώτον, και οι δύο υλοποιήσεις παράγουν βέλτιστο κόστος ταυτόσημο με αυτό της NetworkX σε κάθε περίπτωση που δοκιμάστηκε. Δεύτερον, για μικρές τιμές U ο RHS-Scaling είναι ταχύτερος, καθώς η επιβάρυνση που εισάγουν οι μηχανισμοί σύμπτυξης, αναίρεσης και τελικής μέγιστης ροής δεν αντισταθμίζεται από τη μείωση φάσεων. Τρίτον, καθώς αυξάνεται το U ο ισχυρά πολυωνυμικός γίνεται ταχύτερος (στο $U = 10^6$), τόσο σε πυκνά όσο και σε αραιά δίκτυα, σε συμφωνία με τη θεωρητική ανάλυση.

Κεφάλαιο 5. Συμπεράσματα

5.1 Συμπεράσματα

Η παρούσα εργασία παρουσίασε τη θεωρητική ανάλυση και τη λεπτομερή υλοποίηση του αλγορίθμου ροής ελαχίστου κόστους του James B. Orlin (1993). Υλοποιήθηκαν δύο εκδοχές σε γλώσσα C: η τεχνική κλιμάκωσης των Edmonds-Karp (Section 3 του άρθρου) και ο πλήρης ισχυρά πολυωνυμικός αλγόριθμος με συμπτώξεις (Section 4).

Κατά την υλοποίηση έγινε σαφές πόσο περίπλοκες είναι οι λεπτομέρειες στον χειρισμό των συμπτώξεων και των δυναμικών. Ιδιαίτερα οι μηχανισμοί πλήρους αναγέννησης του Δ , η αναίρεση συμπτώξεων με αποκατάσταση δυναμικών, και ο τελικός υπολογισμός βέλτιστης ροής μέσω μέγιστης ροής, απαιτούν προσεκτικό συντονισμό και αποθήκευση επαρκών στιγμιotypών κατάστασης.

Η χρήση της γεννήτριας σε Python με τη NetworkX ως αναφορά ορθότητας αποδείχθηκε πολύ χρήσιμη: επιτρέπει τον έλεγχο πολλαπλών δικτύων με διαφορετικές παραμέτρους και αποκαλύπτει ενδεχόμενα σφάλματα στην υλοποίηση. Η μέτρηση χρόνων εκτέλεσης επιβεβαιώνει την πρακτική υπεροχή του απλού RHS-Scaling για μικρές τιμές U , αλλά αναδεικνύει και τη σημασία του ισχυρά πολυωνυμικού αλγορίθμου όταν το U γίνει εξαιρετικά μεγάλο.

5.2 Μελλοντικές επεκτάσεις

Αρκετές κατευθύνσεις επέκτασης ανοίγονται. Πρώτον, η αντικατάσταση της γραμμικής υλοποίησης του Dijkstra με μια υλοποίηση που βασίζεται σε binary heap ή σε Fibonacci heap θα βελτιώσει τον χρόνο ανά συντομότερο μονοπάτι και θα φέρει την πρακτική συμπεριφορά πιο κοντά στο θεωρητικό βέλτιστο.

Δεύτερον, η υποστήριξη χωρητικών δικτύων, είτε εσωτερικά στον κώδικα είτε μέσω εξωτερικού μετασχηματισμού που διαχωρίζει κάθε χωρητική ακμή σε νέο κόμβο και δύο μη χωρητικές ακμές, θα διευρύνει το εύρος εφαρμογών. Η υλοποίηση μπορεί να επεκταθεί ώστε να αποδεχθεί και αρχεία εισόδου με χωρητικότητες.

Τρίτον, η βελτίωση της γεννήτριας δοκιμών, ώστε να παράγει δίκτυα με καθορισμένη δομή (π.χ. bipartite matching, transportation problems, planar networks), θα επιτρέψει πιο στοχευμένη πειραματική αξιολόγηση. Επίσης, θα ήταν ενδιαφέρον να συγκριθούν τα αποτελέσματα με άλλους αλγορίθμους (π.χ. Network Simplex, Cost-Scaling), που

διαφορετικά βασίζονται στο ίδιο θεωρητικό υπόβαθρο.

Τέλος, το άρθρο του Orlin αναφέρει ότι ο αλγόριθμος μπορεί να υλοποιηθεί παράλληλα σε χρόνο $O(m \log^3 n)$ με $O(n^3)$ επεξεργαστές. Μια μελλοντική εργασία θα μπορούσε να εξετάσει εμπειρικά την παραλληλοποίηση του Dijkstra στο εσωτερικό του αλγορίθμου.

Βιβλιογραφία

- [1] Orlin, J. B. (1993). A Faster Strongly Polynomial Minimum Cost Flow Algorithm. *Operations Research*, 41(2), 338-350.
- [2] Edmonds, J., & Karp, R. M. (1972). Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems. *Journal of the ACM*, 19, 248-264.
- [3] Tardos, É. (1985). A Strongly Polynomial Minimum Cost Circulation Algorithm. *Combinatorica*, 5, 247-255.
- [4] Galil, Z., & Tardos, É. (1986). An $O(n^2(m + n \log n) \log n)$ Min-Cost Flow Algorithm. In *Proceedings of the 27th Annual Symposium on Foundations of Computer Science*, 136-146.
- [5] Goldberg, A. V., & Tarjan, R. E. (1987). Solving Minimum Cost Flow Problems by Successive Approximation. In *Proceedings of the 19th ACM Symposium on the Theory of Computing*, 7-18.
- [6] Goldberg, A. V., & Tarjan, R. E. (1988). Finding Minimum-Cost Circulations by Canceling Negative Cycles. In *Proceedings of the 20th ACM Symposium on the Theory of Computing*, 388-397.
- [7] Ahuja, R. K., Goldberg, A. V., Orlin, J. B., & Tarjan, R. E. (1988). Finding Minimum Cost Flows by Double Scaling. *Mathematical Programming*, 53, 243-266.
- [8] Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ.
- [9] Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *Journal of the ACM*, 34, 596-615.
- [10] Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1, 269-271.
- [11] Fujishige, S. (1986). An $O(m^3 \log n)$ Capacity-Rounding Algorithm for the Minimum Cost Circulation Problem: A Dual Framework of Tardos' Algorithm. *Mathematical Programming*, 35, 298-309.
- [12] Röck, H. (1980). Scaling Techniques for Minimal Cost Network Flows. In *Discrete Structures and Algorithms*, Carl Hansen, Munich, 101-191.
- [13] Ford, L. R., Jr., & Fulkerson, D. R. (1962). *Flows in Networks*. Princeton University Press.
- [14] NetworkX Developers. NetworkX: Network analysis in Python. <https://networkx.org/>
- [15] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11-15.

Παράρτημα Α'. Μετάφραση ξένων όρων

Ο παρακάτω πίνακας συνοψίζει τους κυριότερους ξένους όρους που χρησιμοποιούνται στην εργασία και τις ελληνικές αποδόσεις τους.

Ελληνικός όρος	Ξένος όρος (Αγγλικός)
Ροή ελαχίστου κόστους	<i>Minimum cost flow</i>
Δίκτυο ροής	<i>Flow network</i>
Ψευδοροή	<i>Pseudoflow</i>
Υπολειπόμενο δίκτυο	<i>Residual network</i>
Μειωμένο κόστος	<i>Reduced cost</i>
Δυναμικό κόμβου	<i>Node potential</i>
Κλιμάκωση δεξιού μέλους	<i>Right-hand side scaling</i>
Ισχυρά πολυωνυμικός αλγόριθμος	<i>Strongly polynomial algorithm</i>
Ισχυρά εφικτή ακμή	<i>Strongly feasible arc</i>
Σύμπτυξη κόμβων	<i>Node contraction</i>
Αναίρεση σύμπτυξης	<i>Uncontraction</i>
Πλήρης αναγέννηση	<i>Complete regeneration</i>
Φάση κλιμάκωσης	<i>Scaling phase</i>
Συντομότερο μονοπάτι	<i>Shortest path</i>
Επαύξηση ροής	<i>Flow augmentation</i>
Ισορροπία μάζας	<i>Mass balance</i>
Πλεόνασμα / Υπερβάλλον	<i>Excess / Imbalance</i>
Πηγή / Καταβόθρα	<i>Source / Sink</i>
Μέγιστη ροή	<i>Maximum flow</i>
Ισχυρά συνδεδεμένο γράφημα	<i>Strongly connected graph</i>
Χωρητικότητα	<i>Capacity</i>
Κόστος ακμής	<i>Arc cost</i>
Υπερ-πηγή / Υπερ-καταβόθρα	<i>Super source / Super sink</i>
Συνθήκη συμπληρωματικής χαλαρότητας	<i>Complementary slackness condition</i>
Τεχνική διπλής κλιμάκωσης	<i>Double scaling technique</i>